SISET

Sistema de Gestión y Seguimiento de Trámites

ESFM “Mcal. Andrés de Santa Cruz y Calahumana” — UA Ancocagua

Respuestas a las observaciones a corregir

Desarrollo, lanzamiento, pruebas y evaluación de calidad (ISO/IEC 25010) y seguridad (ISO/IEC 27001), con evidencias

La Paz, 22 de octubre de 2026

Contenido

TOC \h \o "1-2"HYPERLINK "" \l "_Toc1"Cómo se obtuvieron los datos.....PAGEREF _Toc1 \h
Hoja 1 — Desarrollo incremental: ¿cómo se desarrolló?.....5
Herramientas de desarrollo.....5
Lenguajes de programación utilizados en el proyecto.....5
Frameworks y librerías utilizadas en el desarrollo.....5
Tecnologías web.....6
Servidor web.....6
Hosting y despliegue del sistema.....6
Computación en la nube.....6
Tendencias tecnológicas aplicadas al proyecto.....6
Calidad de software.....7
Seguridad del software.....7
Norma ISO/IEC 27001.....7
Métricas de calidad del software.....7
Norma ISO/IEC 25010.....8
Gestión de seguridad de la información.....8
Copias de seguridad.....8
Gestión de riesgos.....8
Cifrado y protección de información.....8
Modelo COCOMO.....8
Estimación de costos de software.....9
Estimación de tiempo y esfuerzo.....9
Factibilidad técnica.....9
Factibilidad operativa.....9
Factibilidad económica.....9
Rentabilidad del proyecto.....10
Valor Actual Neto (VAN).....10
Tasa Interna de Retorno (TIR).....10
Relación costo beneficio.....10
Pruebas de software.....10
Pruebas funcionales.....11
Pruebas de compatibilidad.....12
Pruebas de seguridad.....12
Hoja 1 — Fase de lanzamiento: ¿cómo se realizó?.....13
Despliegue del sistema.....13
Configuración del servidor.....13
Publicación del sistema.....13
Capacitación de usuarios.....13
Documentación final.....13
Puesta en producción.....14
Hoja 1 — Fase de pruebas: ¿se realizó o no se realizó aún?.....15
4.2. Evaluación de calidad del software — ISO/IEC 25010.....16
4.2.1. Evaluación de adecuación funcional.....16
4.2.2. Evaluación de eficiencia de desempeño.....17
4.2.3. Evaluación de compatibilidad.....22
4.2.4. Evaluación de usabilidad.....24
4.2.5. Evaluación de fiabilidad.....26
4.2.6. Evaluación de seguridad.....28
4.2.7. Evaluación de mantenibilidad.....30
4.2.8. Evaluación de portabilidad.....30
4.3. Evaluación de seguridad — ISO/IEC 27001.....32
4.3.1. Evaluación de riesgos.....32
4.3.2. Evaluación de copias de seguridad.....33
4.3.3. Evaluación de seguridad de acceso.....34
4.3.4. Evaluación de vulnerabilidades.....35

4.4. Resultados generales....36
Tabla consolidada de métricas....36
Gráficos comparativos....36
Cumplimiento de ISO 25010....36
Cumplimiento de ISO 27001....37
Interpretación global de resultados....37
4.5. Conclusiones del capítulo....38
Nivel de calidad alcanzado....38
Nivel de seguridad alcanzado....38
Fortalezas del sistema....38
Aspectos a mejorar....38
Validación del cumplimiento de estándares ISO....38
Glosario de términos técnicos....39
Anexo — Cómo repetir las pruebas....40

(Si el índice aparece vacío, en Word: clic derecho sobre él → Actualizar campos.)

Cómo se obtuvieron los datos

Este documento responde, punto por punto, las dos hojas del archivo **“observaciones a corregir.xlsx”**: la Hoja 1 (cómo se desarrolló el sistema, la fase de lanzamiento y la fase de pruebas) y la Hoja 2 (evaluación de calidad ISO/IEC 25010, evaluación de seguridad ISO/IEC 27001, resultados generales y conclusiones). Cada fórmula del archivo se calculó con **datos medidos el 22 de octubre de 2026**, no con valores supuestos, salvo en la parte económica, donde los supuestos se marcan.

Se usaron dos entornos:

- **Stack local en Docker**, idéntico al de producción (PostgreSQL 18, API Node.js/Express, web con Nginx), con los datos de prueba del proyecto. Aquí se hicieron las pruebas de carga, estrés, integridad, seguridad, compatibilidad y respaldo, para no afectar al servidor compartido.
- **Servidor de prueba** https://siset.vertexall.com (VPS con Coolify, solo datos ficticios). Aquí solo se hicieron consultas livianas: disponibilidad, cabeceras de seguridad, tiempo en línea y bitácora de accesos.

Herramientas de medición: **k6** (carga y estrés) enviando métricas a **InfluxDB** y graficadas en **Grafana**; un recolector propio de CPU y RAM de cada contenedor; **Playwright** (pruebas de punta a punta y compatibilidad en Chromium, Google Chrome y Firefox); **Lighthouse** (accesibilidad); **Vitest** (pruebas unitarias e integración); **psql**, **pg_dump** y **pg_restore** (integridad y respaldos).

Prueba	Resultado principal	Sección
Carga (50 usuarios)	36 796 peticiones · 0 % de error · p95 = 9,35 ms	4.2.2
Estrés (hasta 400 usuarios)	Quiebre en ~60 usuarios sin pausa por Nginx; tras la corrección, 384 349 peticiones con 0 % de error	4.2.2
Integridad (300 altas simultáneas)	0 números repetidos, 0 saltos, 300/300 eventos en la bitácora	4.2.5
Disponibilidad del servidor	80/80 respuestas correctas · 44 h en línea sin reinicios · 0 errores 5xx	4.2.5
Seguridad	17/17 controles probados OK · 0 vulnerabilidades en dependencias	4.2.6 y 4.3
Compatibilidad	9/9 combinaciones (3 navegadores × 3 tamaños de pantalla)	4.2.3
Accesibilidad (Lighthouse)	98, 100 y 100 sobre 100	4.2.4
Pruebas automatizadas	API 1 121/1 121 (721 unitarias + 400 de integración) · web 265/265 · E2E 103/104	Hoja 1
Respaldo y restauración	18/18 tablas idénticas tras restaurar	4.3.2

Hoja 1 — Desarrollo incremental: ¿cómo se desarrolló?

SISET se construyó con un **desarrollo incremental por sprints** (S0 a S6), del 12 al 20 de octubre de 2026 (218 commits). Cada incremento entregaba una parte usable y probada: S0 base técnica, S1 seguridad y usuarios, S2 files académicos y requisitos, S3 hojas de ruta, S4 seguimiento público, S5 reportes, S6 calidad y despliegue. Cada cambio pasa por una **integración continua** (formato, lint, tipos, pruebas, build, auditoría de dependencias, base de datos, E2E y Docker) antes de quedar en la rama principal. Las respuestas de la escuela se registran como decisiones (64 decisiones registradas y 12 decisiones de arquitectura).

Herramientas de desarrollo

Visual Studio Code

Editor principal del código, con extensiones de TypeScript, ESLint, Prettier, Tailwind CSS y Prisma. El asistente de IA **Claude Code** se usó desde la terminal y el editor para generar, revisar y probar código.

Git y GitHub

Control de versiones con Git en un **monorepo** (apps/web y apps/api) alojado en GitHub. Cada commit lleva el número de la tarea y lo que hace. **Husky** y **lint-staged** revisan formato y lint antes de cada commit. **GitHub Actions** corre la integración continua y publica las imágenes Docker en **GitHub Container Registry**.

Postman

No se usó Postman. Su función la cubren dos herramientas libres: **Swagger UI / OpenAPI** (la API publica su documentación interactiva en desarrollo, generada desde los esquemas Zod) y **Supertest + Vitest**, que prueban automáticamente cada endpoint. Para pruebas manuales se usó **curl**.

Docker

Todo el sistema corre en contenedores: **db** (PostgreSQL), **migrar** (aplica migraciones y termina), **api** (Node.js) y **web** (Nginx que sirve la aplicación y hace de proxy de /api). El mismo docker-compose sirve para desarrollo, CI y el servidor. Los contenedores corren endurecidos: sin privilegios (cap_drop ALL), no-new-privileges y sistema de archivos de solo lectura en la API.

Figma

No se usó Figma. El diseño se hizo **directamente en código** con una guía de diseño propia (**DESIGN.md**, formato Google Stitch): paleta de dos azules (#1F4E79 y #2E75B6), tipografía IBM Plex, componentes con Tailwind CSS e íconos Lucide. Se revisó con capturas de Playwright en escritorio y celular. Los diagramas se hicieron con Mermaid.

Lenguajes de programación utilizados en el proyecto

TypeScript 6 en modo estricto (sin “any”) para el frontend y el backend, sobre **Node.js 26**. **SQL** (PostgreSQL) en las migraciones y en los controles de la base. **HTML y CSS** (Tailwind) en la interfaz. **Bash** en los scripts de despliegue y pruebas.

Frameworks y librerías utilizadas en el desarrollo

Capa	Frameworks y librerías
Frontend	React 19, Vite 8, Tailwind CSS 4, React Router 8, TanStack Query, Axios, Chart.js,

Capa	Frameworks y librerías
	qrcode.react, date-fns, Lucide
Backend	Express 5, Prisma 7 (ORM), Zod 4 (validación), Casbin (permisos), jose (JWT), bcryptjs, Helmet, CORS, express-rate-limit, multer, pino (logs), swagger-ui-express
Pruebas	Vitest, Supertest, Playwright, cobertura v8
Calidad	ESLint, Prettier, Husky, lint-staged, TypeScript estricto

Tecnologías web

Aplicación de una sola página (**SPA**) en React que consume una **API REST** con JSON. Autenticación con **JWT** de 15 minutos y token de renovación en **cookie httpOnly, Secure y SameSite=Strict**. Documentación de la API con **OpenAPI 3**. **Diseño responsivo** desde 360 px. **Códigos QR** en la carátula de la hoja de ruta para el seguimiento público. Seguridad HTTP con Content-Security-Policy, X-Frame-Options y HSTS.

Servidor web

Nginx 1.30 sirve la aplicación web compilada y reenvía /api a la API (**Node.js + Express 5**). En el servidor, **Traefik** (el que trae Coolify) termina HTTPS con certificado **Let's Encrypt** y redirige http → https.

Hosting y despliegue del sistema

VPS propio (Contabo) administrado con **Coolify**. Las imágenes se construyen en GitHub Actions y se publican en GitHub Container Registry con dos etiquetas ("prueba" y "AAAA.MM.DD-sha"), así se puede volver a una versión anterior. El dominio es **siset.vertexall.com** (DNS en Cloudflare). Hoy funciona como **entorno de prueba con datos ficticios**; la puesta en producción está pendiente (ver Fase de lanzamiento).

Computación en la nube

Se usa un modelo **IaaS** (un servidor virtual en la nube de Contabo) y servicios **SaaS** de apoyo: GitHub (código, CI/CD y registro de imágenes), Cloudflare (DNS) y Let's Encrypt (certificados). La aplicación es portable: no depende de ningún servicio propietario de un proveedor y puede moverse a otro VPS o a un servidor de la escuela con el mismo docker-compose.

Tendencias tecnológicas aplicadas al proyecto

- **Desarrollo asistido por inteligencia artificial** (Claude Code) para programar, revisar y probar.
- **Contenedores** (Docker) y **CI/CD** con GitHub Actions.
- **Monolito modular**: un solo despliegue, módulos con límites claros.
- **Seguridad desde el diseño**: RBAC con Casbin, bitácora inmutable y mínimo privilegio en la base.
- **Observabilidad**: logs estructurados (pino) y métricas en Grafana durante las pruebas.
- **Diseño guiado por documentos** (un documento de diseño por tarea, decisiones de arquitectura y registro de decisiones).
- **QR** para seguimiento ciudadano sin necesidad de cuenta.

Calidad de software

La calidad se asegura con una **lista de verificación de "terminado"** por tarea y con una integración continua que bloquea el cambio si falla: formato (Prettier), lint (ESLint, 0 advertencias), tipos (TypeScript estricto), pruebas unitarias e integración (Vitest), migraciones y seed dos veces (idempotencia), pruebas de base de datos con cobertura de seguridad ≥ 70 %, E2E (Playwright), build, auditoría de dependencias y prueba de arranque del sistema completo en Docker. Los atributos se miden con ISO/IEC 25010 en la Hoja 2.

Seguridad del software

- Contraseñas con **bcrypt costo 12**; bloqueo de la cuenta tras **5 intentos** fallidos; límite de **10 logins por minuto** por IP.
- Sesión con **JWT de 15 min** + renovación en cookie httpOnly; detección de **reuso** del token de renovación.
- **Permisos por rol** (Casbin) y cada usuario ve solo lo suyo y lo de su sede.
- **Validación de toda entrada** con Zod (rechaza campos extra y cuerpos > 100 KB).
- **Helmet**, CSP, CORS restringido, HTTPS obligatorio.
- **Bitácora** de toda escritura y acceso en la misma transacción; la tabla no admite UPDATE ni DELETE desde la aplicación.
- Contenedores sin privilegios y **npm audit** en cada integración.

Norma ISO/IEC 27001

Se tomaron como referencia los controles del **Anexo A de ISO/IEC 27001:2022** que aplican a un sistema de este tamaño. El cumplimiento medido está en la sección 4.3.

Control (Anexo A)	Cómo lo cumple SISET	Estado
5.15 Control de acceso	Permisos por rol (Casbin); cada usuario ve solo lo suyo y lo de su sede	Cumple
5.17 Información de autenticación	bcrypt 12, contraseña temporal con cambio obligatorio	Cumple
8.2 Derechos de acceso privilegiado	Rol ADMIN separado (una cuenta aparte para administrar)	Cumple
8.5 Autenticación segura	Bloqueo, rate limit, JWT corto; falta TOTP	Parcial
8.13 Respaldo de la información	Restauración probada; falta el respaldo diario automático	Parcial
8.15 Registro (logging)	Bitácora inmutable + logs pino	Cumple
8.20 Seguridad de redes	HTTPS, CORS, contenedores en red interna	Cumple
8.24 Uso de criptografía	TLS, bcrypt, JWT firmado; sin cifrado en reposo	Parcial
8.25 Ciclo de desarrollo seguro	Documento de diseño por tarea, revisión y pruebas de seguridad automáticas	Cumple
8.28 Codificación segura	Zod, Prisma (sin SQL concatenado), TS estricto	Cumple
8.32 Gestión de cambios	Git, CI, imágenes versionadas, rollback	Cumple

Métricas de calidad del software

Se usaron las fórmulas del archivo (Hoja 2): adecuación funcional, tiempo promedio, uso de CPU, compatibilidad, usabilidad, disponibilidad, seguridad, mantenibilidad y portabilidad; y para

seguridad: riesgo, tasa de recuperación, accesos seguros y corrección de vulnerabilidades. Además, cobertura de pruebas (98,55 % de sentencias en el módulo de seguridad) y percentiles de tiempo de respuesta (p95).

Norma ISO/IEC 25010

El modelo de calidad del producto ISO/IEC 25010 guía los requisitos no funcionales del proyecto (seguridad, desempeño, usabilidad, disponibilidad, etc.) y la evaluación de la Hoja 2: adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad. Resultado global: **95,18 %** (sección 4.4).

Gestión de seguridad de la información

- **Mínimo privilegio**: cada rol ve solo lo suyo; la aplicación se conecta a la base con un rol sin permisos para borrar la bitácora.
- **Separación de entornos**: el servidor de prueba solo tiene datos ficticios; los datos reales (por ejemplo, fotos del folder personal) no se suben a ningún servicio.
- **Trazabilidad**: quién, cuándo, desde qué IP y qué cambió (antes/después).
- **Secretos** fuera del código (variables de entorno generadas por Coolify).
- **Gestión de riesgos** con matriz de probabilidad × impacto (sección 4.3.1).

Copias de seguridad

Se probó el procedimiento completo de respaldo y restauración (pg_dump → pg_restore en una base nueva): **18 de 18 tablas idénticas** y la misma huella MD5 (sección 4.3.2). **Pendiente antes de producción**: programar el respaldo **diario** automático con retención de 30 días y guardarlo fuera del servidor.

Gestión de riesgos

Se identificaron 13 riesgos y se valoró cada uno con **Riesgo = Probabilidad × Impacto** (escala 1 a 5): 1 alto, 7 medios y 5 bajos. El detalle, la matriz y los controles están en la sección 4.3.1.

Cifrado y protección de información

- **En tránsito**: HTTPS con TLS (Let’s Encrypt) y redirección automática desde http.
- **Contraseñas**: hash bcrypt de costo 12 (nunca se guardan ni se registran en texto plano).
- **Sesiones**: JWT firmado (HS256) de 15 minutos y token de renovación en cookie httpOnly, Secure y SameSite=Strict.
- **En reposo**: la base no está cifrada a nivel de disco. Se recomienda cifrar el volumen y los respaldos antes de producción.

Modelo COCOMO

Se aplicó **COCOMO básico, modo orgánico** (equipo pequeño, requisitos conocidos), con el tamaño real del código fuente: **27,07 KLOC** (6 545 líneas de la API, 19 815 de la web y 714 del modelo de datos, sin contar pruebas ni código generado).

Ecuación	Cálculo	Resultado
Esfuerzo E = 2,4 × KLOC^1,05	2,4 × 27,07^1,05	76,63 personas-mes
Tiempo T = 2,5 × E^0,38	2,5 × 76,63^0,38	13,00 meses
Personal P = E / T	76,63 / 13,00	5,89 personas

> Interpretación: COCOMO estima cuánto costaría hacer este sistema con un desarrollo tradicional. El proyecto real se hizo en **9 días de trabajo (12 al 20 de octubre) con 2 personas y asistencia de IA**, muy por debajo de lo que estima el modelo. La diferencia muestra el efecto de la IA y de la reutilización de librerías. COCOMO (de 1981) no la considera.

Estimación de costos de software

Con un costo de **Bs 6.000 por persona-mes** (supuesto: sueldo de un programador en Bolivia, con cargas), el costo según COCOMO sería **76,63 × 6.000 = Bs 459.772**. El costo real del proyecto se estima en **Bs 17.000** (ver Factibilidad económica).

Estimación de tiempo y esfuerzo

Según COCOMO: **76,63 personas-mes** en **13,00 meses** con unas **5,89 personas**. Real: 218 commits entre el 12 y el 20 de octubre de 2026; el backlog del 12 de octubre sumaba 301 horas y el 14 de octubre ya iba en 41 % (123 h).

Factibilidad técnica

Factible. Todo el software es libre (requisito del proyecto) y estable; corre en un VPS común. La prueba de carga mostró que con 50 usuarios simultáneos la API usa en promedio 21 % de un núcleo y 175 MB de RAM, y la base 65 MB. El equipo domina la pila (TypeScript, React, Node.js, PostgreSQL, Docker).

Factibilidad operativa

Factible. El personal (Kardex, secretarías y coordinación) usa navegador y celular, sin instalar nada. La interfaz cumple accesibilidad 98–100 (Lighthouse) y funciona en celular de 360 px. Hay un manual en /guia/manual y una capacitación programada para el **28 y 29 de octubre**. El estudiante consulta su file y el seguimiento con el número de trámite y su CI, o con el QR.

Factibilidad económica

> Los valores de esta sección son **supuestos razonables** del equipo, no datos de la escuela. Para la versión final conviene reemplazarlos con los montos reales (sueldos, horas ahorradas, costo del servidor).

Concepto	Supuesto	Monto (Bs)
Inversión inicial (I_0)	2 desarrolladores × 2 meses × Bs 4 000 + Bs 1 000 de capacitación y materiales	17.000
Costo anual de operación	VPS Bs 70/mes + dominio Bs 100/año + mantenimiento 8 h/mes × Bs 50	5.740
Beneficio anual	4 funcionarios × 5 h/semana ahorradas × 40 semanas × Bs 25/h + Bs 2 000 de papel y fotocopias	22.000
Flujo neto anual	Beneficio − costo de operación	16.260
Tasa de descuento / horizonte	10 % anual / 5 años	—

Rentabilidad del proyecto

Con esos supuestos el proyecto es **rentable**: el VAN es positivo, la TIR supera a la tasa de descuento y la relación beneficio/costo es mayor que 1. La inversión se recupera en unos **12,5 meses**. El beneficio principal no es solo dinero: menos tiempo de atención, trámites rastreables y menos pérdida de documentos.

Valor Actual Neto (VAN)

$VAN = -I_0 + \Sigma\ Flujo_t / (1 + i)^t = -17.000 + 16.260 \times (0{,}9091 + 0{,}8264 + 0{,}7513 + 0{,}6830 + 0{,}6209)$ = **Bs 44.638**. Como VAN > 0, el proyecto genera valor.

Tasa Interna de Retorno (TIR)

La TIR es la tasa que hace VAN = 0: **92,0 %** anual, muy por encima del 10 % exigido.

Relación costo beneficio

$B/C = VA(beneficios) / (I_0 + VA(costos))$ = **2,15**. Por cada boliviano invertido se recuperan Bs 2,15.

Pruebas de software

Se aplicaron pruebas en cuatro niveles. Todas se pueden repetir con los scripts del proyecto.

Nivel	Herramienta	Resultado
Unitarias (API)	Vitest	721 de 721
Unitarias y componentes (web)	Vitest	265 de 265
Integración API + PostgreSQL	Vitest + Supertest	400 de 400 (corrida completa de la API: 1 121 de 1 121) · cobertura de seguridad 98,55 %
Funcionales de punta a punta	Playwright	103 de 104 (1 prueba intermitente)
Rendimiento (carga y estrés)	k6 + Grafana	Ver 4.2.2
Seguridad	Script propio + npm audit	17 de 17 · 0 vulnerabilidades
Compatibilidad	Playwright	9 de 9

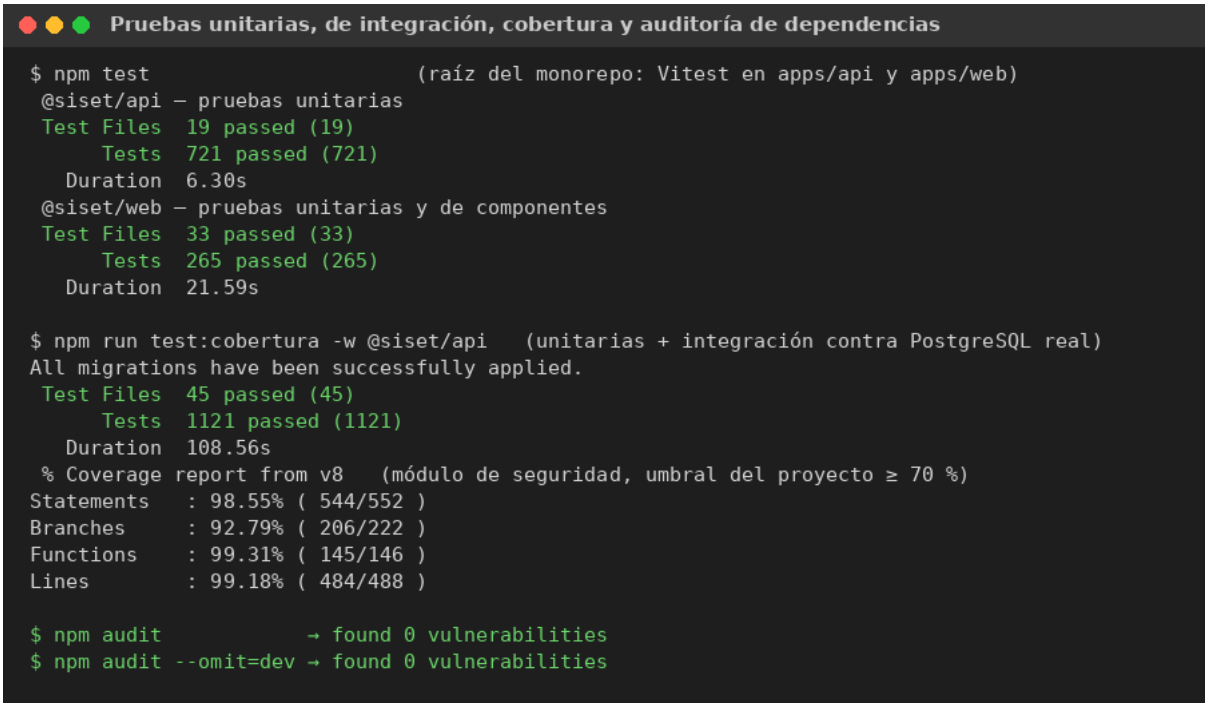


Figura 1. *Pruebas unitarias, de integración con PostgreSQL, cobertura y auditoría de dependencias.*

Pruebas funcionales

Las pruebas E2E con Playwright recorren los flujos reales de cada rol: login, permisos, files, requisitos, bloqueo, hojas de ruta (crear, derivar, firmar, imprimir, devolver, entregar), seguimiento público, bitácora y usuarios. Corren en tres modos: con datos de maqueta (mock), contra la API y contra **la API y PostgreSQL de verdad** (api-real). Resultado: **103 de 104**. La que falló es intermitente (pasa 2 de cada 3 veces) y quedó registrada para corregir.

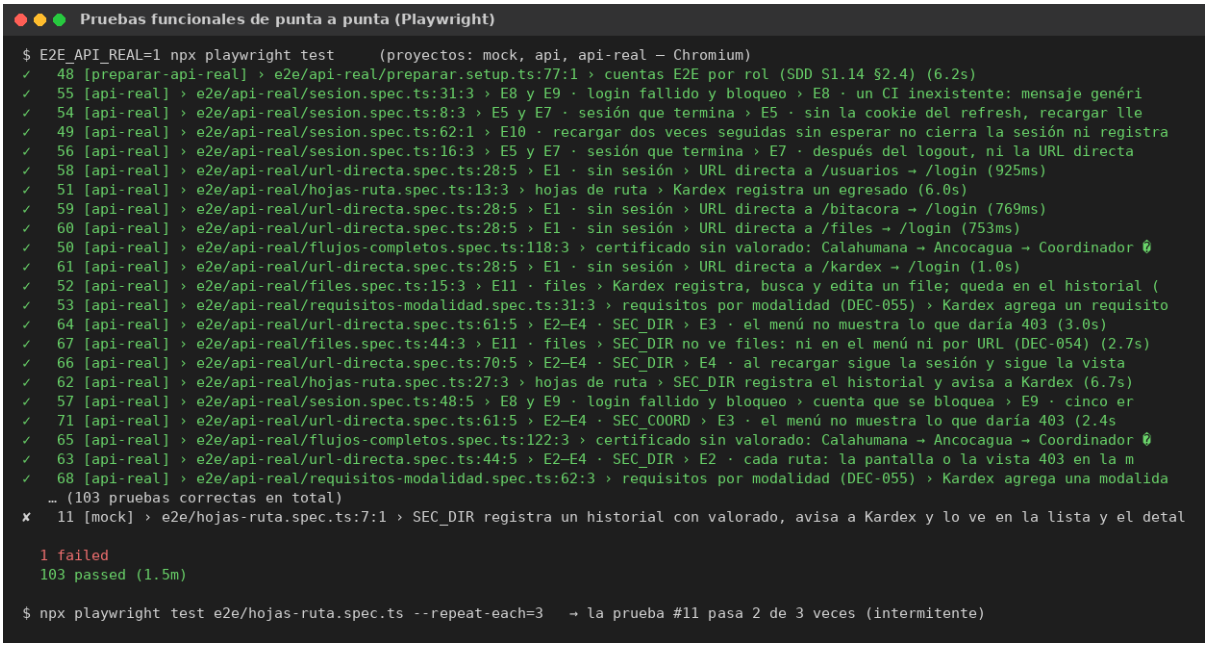


Figura 2. *Pruebas funcionales de punta a punta: 103 correctas; la prueba intermitente se repitió 3 veces.*

Pruebas de compatibilidad

Ver la sección 4.2.3: 9 de 9 combinaciones exitosas (Chromium, Google Chrome y Firefox en escritorio, tablet y celular).

Pruebas de seguridad

Ver las secciones 4.2.6 y 4.3.3–4.3.4: 17 de 17 controles probados, 0 vulnerabilidades en dependencias.

Hoja 1 — Fase de lanzamiento: ¿cómo se realizó?

Despliegue del sistema

Con el flujo **“Publicar imágenes”** de GitHub Actions se construyen las imágenes siset-api, siset-migrar y siset-web y se publican en GitHub Container Registry. En el servidor, Coolify baja las imágenes y levanta el servicio: primero **migrar** aplica las migraciones de la base (y los datos de prueba), luego arrancan **api** y **web** cuando la base está sana. Cada actualización repite el proceso; para volver atrás basta con cambiar la etiqueta de la imagen.

Configuración del servidor

- VPS Contabo con Docker y **Coolify** (panel y API de despliegue).
- **Traefik** con certificado **Let’s Encrypt** automático y redirección a HTTPS; DNS en Cloudflare (solo DNS).
- PostgreSQL con límites de recursos (max_connections=30, shared_buffers=128 MB) y volumen persistente.
- API con rol de base de mínimo privilegio, secretos generados por Coolify, healthchecks y reinicio automático.

Publicación del sistema

Publicado como **entorno de prueba** en **https://siset.vertexall.com** desde el 14 de octubre de 2026, con cuentas de prueba para cada rol (listadas en el login). Se redesplegó con cada cambio importante; la última versión lleva **44 horas en línea sin reinicios**.

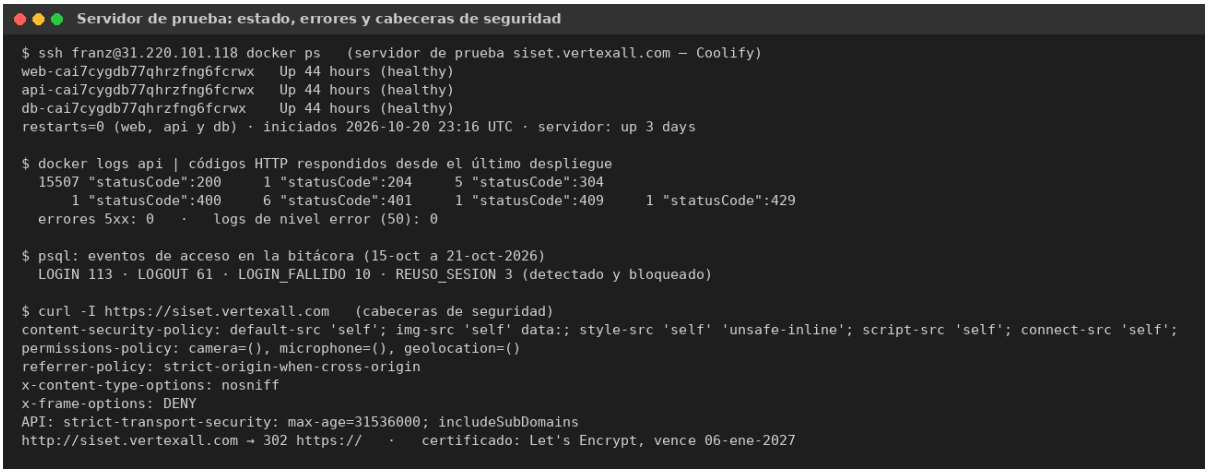


Figura 3. *Servidor de prueba: contenedores sanos, 0 reinicios, 0 errores 5xx, accesos registrados y cabeceras de seguridad.*

Capacitación de usuarios

Programada para el 28 y 29 de octubre de 2026. Material listo en el propio sistema: **manual del personal y guía de capacitación** en /guia/manual y guía de estudio. Pendiente: confirmar el horario con la escuela.

Documentación final

- Documento de requisitos, alcance, reglas de negocio y modelo de datos.
- 12 decisiones de arquitectura y 64 decisiones del proyecto, cada una con su fuente.
- Documento de diseño por tarea, guía de despliegue y guía de diseño visual.

- Manual del usuario y guía de capacitación dentro del sistema; documentación de la API en OpenAPI.
- Este informe de pruebas y evaluación.

Puesta en producción

Aún no realizada. El sistema está completo para el piloto, pero antes de la puesta en producción faltan: (1) el respaldo diario automático; (2) los datos del Administrador real; (3) aplicar las dos correcciones encontradas en estas pruebas (keepalive en Nginx y HSTS en la web); (4) la capacitación del 28–29 de octubre.

Hoja 1 — Fase de pruebas: ¿se realizó o no se realizó aún?

Prueba	¿Se realizó?	Evidencia
Pruebas unitarias	SÍ	API 721/721 y web 265/265 (Vitest), en cada integración continua.
Pruebas de integración	SÍ	400/400 contra PostgreSQL real (1 121/1 121 con las unitarias); cobertura de seguridad 98,55 %. Integridad con 300 altas simultáneas.
Pruebas funcionales	SÍ	E2E Playwright 103/104 (mock, API y api-real). Adecuación funcional en 4.2.1.
Pruebas de compatibilidad	SÍ	9/9 (Chromium, Chrome y Firefox × escritorio, tablet y celular). Safari pendiente: requiere una Mac o un iPhone.
Pruebas de rendimiento	SÍ	Carga con 50 usuarios (p95 = 9,35 ms, 0 % de error) y estrés hasta 400 usuarios, con Grafana.
Pruebas de seguridad	SÍ	17/17 controles, npm audit sin vulnerabilidades, cabeceras y TLS revisados.
Corrección de errores	SÍ, en curso	Fallas de CI: 4 de 4 rachas corregidas (mediana de 17 min). Hallazgos de hoy: keepalive de Nginx (corrección verificada), HSTS en la web y 1 E2E intermitente (pendientes).

4.2. Evaluación de calidad del software — ISO/IEC 25010

4.2.1. Evaluación de adecuación funcional

Métrica aplicada

Grado en que las funciones cumplen los 14 requisitos funcionales del documento de requisitos del proyecto y grado en que los flujos funcionales probados de punta a punta terminan correctamente.

Fórmula utilizada

$$Adecuación\ Funcional = \frac{Funciones\ Correctas}{Funciones\ Evaluadas} \times 100$$

Por pruebas: $103 / 104 \times 100$ · Por requisitos: $10 / 14 \times 100$

Resultado: 99,04 % (pruebas) · 71,43 % (requisitos completos)

Tabla de funcionalidades evaluadas

N.º	Requisito funcional	Estado	Observación
1	Autenticación y cierre de sesión	Parcial	Inicio de sesión, bloqueo, vencimiento y cierre de sesión OK; falta la doble verificación por código (TOTP)
2	Control de acceso por roles	Cumple	403 al entrar por URL directa (probado)
3	Gestión de usuarios	Cumple	Rol ADMIN; baja lógica; CI único
4	Consulta del file académico	Cumple	Mi file del estudiante y búsqueda de Kardex
5	Registro del TCLO	Cumple	Se registra como requisito del folder personal del estudiante
6	Recepción física del certificado TACFI	Cumple	Con curso y entidad
7	Alertas y bloqueo por requisitos	Cumple	Semáforo y bloqueo con resolución en PDF
8	Registro de la hoja de ruta (trámite)	Cumple	Correlativo único, QR y carátula
9	Seguimiento público del trámite	Cumple	Con N.º de trámite + CI o QR
10	Pases entre sedes	Cumple	Bandeja “por recibir”
11	Vista previa de documentos	Parcial	Plantillas oficiales pendientes
12	Impresión oficial	Parcial	Registra N.º de valorado; plantillas pendientes
13	Firma del Coordinador, reportes y panel	Parcial	Firmas y reportes OK; falta exportar a PDF/Excel
14	Bitácora de auditoría	Cumple	Inmutable; filtro por fecha, usuario y módulo

Resultados obtenidos

10 requisitos se cumplen completos y 4 parcialmente; ninguno falta del todo. Si los parciales cuentan como medio cumplimiento, la adecuación es (10 + 4 × 0,5) / 14 = **85,71 %**. De los 104 flujos E2E, 103 terminan bien.

Porcentaje de cumplimiento

99,04 % en pruebas funcionales; **71,43 %** de requisitos completos; **85,71 %** ponderado.

Interpretación de resultados

Lo que está construido funciona correctamente (99 %). Lo que falta para el 100 % de los requisitos depende de respuestas de la escuela (plantillas oficiales) o de funciones planificadas (TOTP y exportar reportes). Ninguna función crítica del flujo diario (files, hojas de ruta, seguimiento, permisos) está incompleta.

Ejemplo de fórmula

Adecuación funcional = Funciones correctas / Funciones evaluadas × 100 = 103 / 104 × 100 = 99,04 %.

4.2.2. Evaluación de eficiencia de desempeño

Tiempo de respuesta del sistema

Prueba de **carga** con k6: de 0 a 50 usuarios simultáneos en 6 minutos (cada usuario recorre 9 pantallas y espera 1 a 3 s, como una persona real). Se hicieron **36 796 peticiones**.

$$Tiempo\ Promedio = \frac{\sum Tiempo\ de\ Respuesta}{Número\ de\ Pruebas}$$

$$\text{Tiempo promedio} = 126\ 966\ \text{ms} \ / \ 36\ 796 = 3{,}45\ \text{ms}$$

Resultado: 3,45 ms (p95 = 9,35 ms; máximo 269 ms)

Consumo de recursos

$$Uso\ de\ CPU = \frac{CPU\ Utilizado}{CPU\ Disponible} \times 100$$

$$\text{CPU del equipo durante la carga: } 6{,}4\ \%\ \text{utilizado} \ / \ 100\ \%\ \text{disponible} \times 100$$

Resultado: 6,4 % en promedio (pico 23 %)

Por contenedor (porcentaje de **1 núcleo**): API 21,4 % (pico 54 %) y 175 MB de RAM; base de datos 8,9 % y 65 MB; web 3,8 % y 15 MB.

Resultados de pruebas

Prueba	Usuarios	Peticiones	Error	Promedio	p95	Meta (< 2 s)
Carga	0 → 50	36 796	0,00 %	3,45 ms	9,35 ms	p95 < 2 s
Estrés (antes)	0 → 400	590 341	57,03 %	114 ms	464 ms	Quiebre a ~60 usuarios
Estrés (después)	0 → 400	384 349	0,00 %	194 ms	633 ms	p95 < 2 s
Integridad (escritura)	30	300 altas	0,00 %	118 ms	209 ms	—

Fórmulas

Tiempo promedio = Σ tiempo de respuesta / número de pruebas. Uso de CPU = CPU utilizado / CPU disponible × 100. Se agregan los percentiles p95 (el 95 % de las peticiones tarda menos) porque el promedio esconde los casos lentos.

Contenido — Tabla de tiempos

Operación	Peticiones	Promedio (ms)	Mediana (ms)	p95 (ms)	Máx (ms)
POST /auth/login	4	262,65	261,10	269,00	269,00
GET /files	4.088	6,40	5,24	12,20	34,61
GET /files/:id	4.088	4,76	3,66	10,03	35,02
GET /files/mio	4.088	4,78	4,01	9,95	24,79
GET /reportes/hojas-ruta	4.088	4,83	3,95	9,95	46,41
GET /hojas-ruta	4.088	4,23	3,26	9,00	41,69
GET /files/:id/requisitos	4.088	2,14	1,61	4,67	19,20
GET /hojas-ruta/tipos	4.088	1,95	1,48	4,38	17,24
GET /auth/mis-permisos	4.088	1,19	0,93	2,57	8,66
GET / (página)	4.084	0,53	0,49	0,94	44,46

El login tarda unos 260 ms **a propósito**: bcrypt de costo 12 hace lento cada intento para frenar a quien prueba contraseñas.

Gráficos de rendimiento

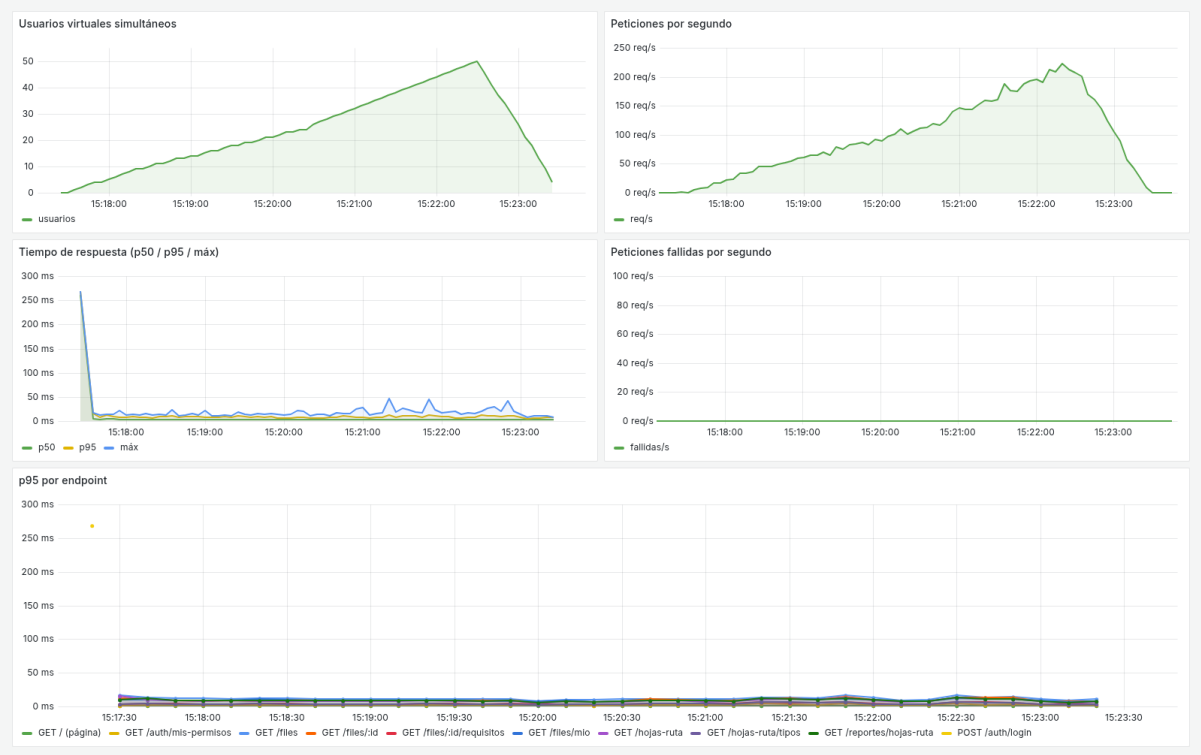


Figura 4. *Grafana — prueba de carga: usuarios, peticiones por segundo, tiempos de respuesta y errores (0).*

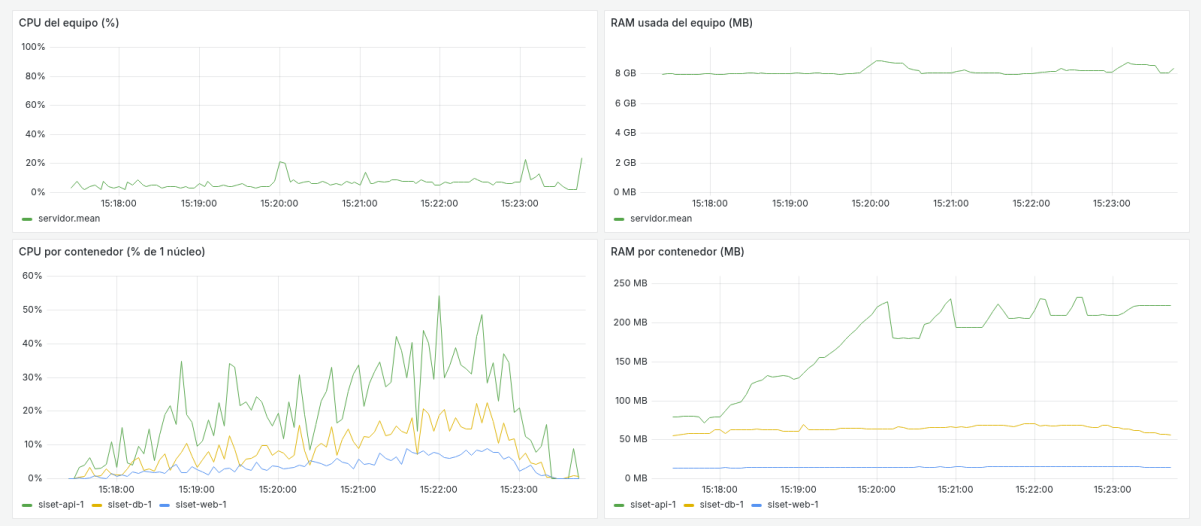


Figura 5. *Grafana — consumo de CPU y RAM del equipo y de cada contenedor durante la carga.*

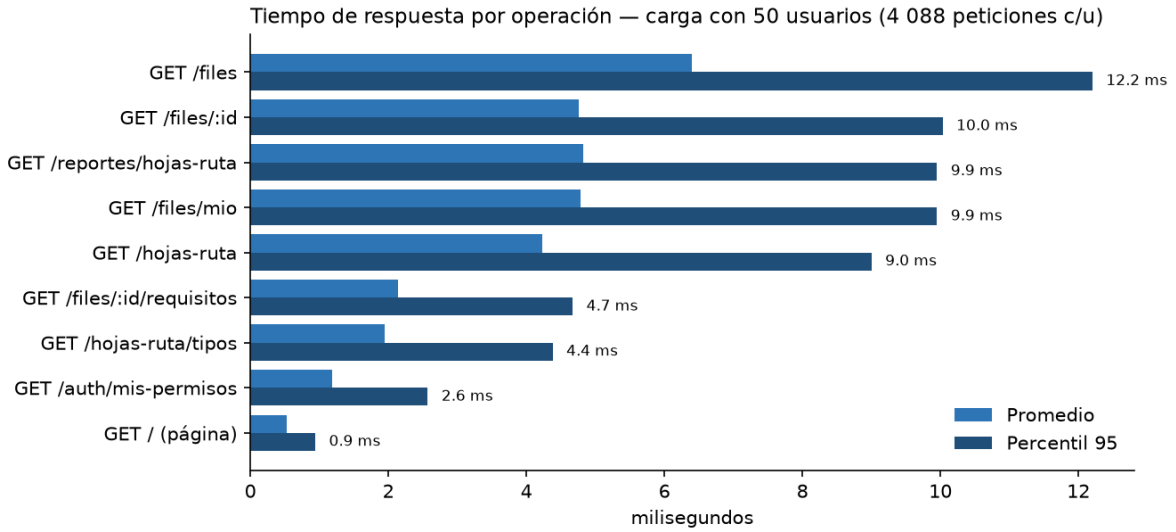


Figura 6. *Tiempo promedio y p95 por operación con 50 usuarios.*

```
k6 run carga.js — 50 usuarios simultáneos (resumen)

  THRESHOLDS
    http_req_duration
    ✓ 'p(95)<500' p(95)=9.35ms
    http_req_failed
    ✓ 'rate<0.01' rate=0.00%
  TOTAL RESULTS
    checks_total.......: 4088    11.31545/s
    checks_succeeded...: 100.00% 4088 out of 4088
    checks_failed......: 0.00%   0 out of 4088
    ✓ todas 200
    HTTP
    http_req_duration..............: avg=3.45ms min=202.96µs med=2.85ms max=268.99ms p(90)=7.33ms p(95)=9.35ms
      { expected_response:true }...: avg=3.45ms min=202.96µs med=2.85ms max=268.99ms p(90)=7.33ms p(95)=9.35ms
    http_req_failed................: 0.00%  0 out of 36796
    http_reqs......................: 36796  101.850121/s
    EXECUTION
    iteration_duration.............: avg=2.03s  min=1.01s    med=2.02s  max=3.05s    p(90)=2.84s  p(95)=2.94s
    iterations.....................: 4088   11.31545/s
    vus............................: 3      min=0          max=50
    vus_max........................: 50     min=50         max=50
    NETWORK
```

Figura 7. *Resumen de k6 de la prueba de carga: umbrales cumplidos y 100 % de verificaciones correctas.*

Prueba de estrés. Se subió a 400 usuarios **sin pausa** (cada uno manda peticiones continuamente) para hallar el límite. Desde ~60 usuarios (~950 peticiones por segundo)

aparecieron errores 502. El log de Nginx mostró la causa: **“connect() failed (99: Address not available)”**. Nginx abría una conexión TCP nueva hacia la API en cada petición y agotó los puertos del contenedor. **La API nunca se cayó** (0 reinicios, siempre sana).

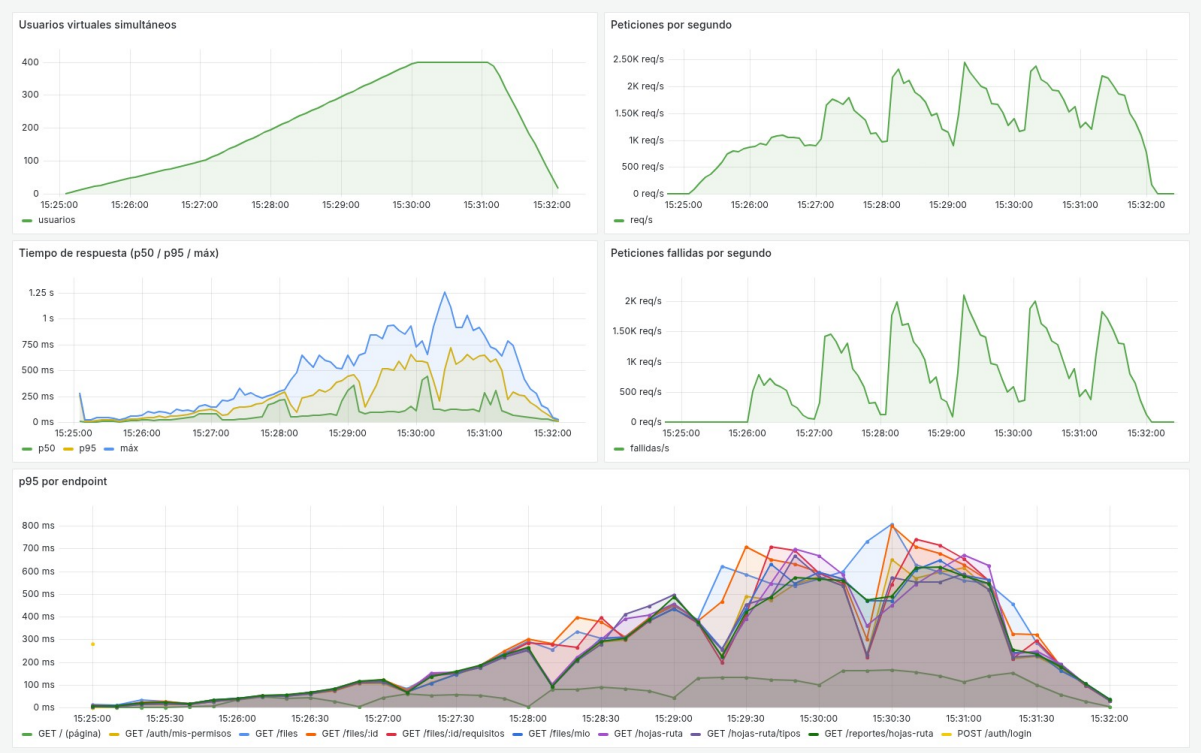


Figura 8. *Grafana — estrés ANTES de la corrección: las peticiones fallidas aparecen desde ~60 usuarios.*

```
k6 run estres.js — hasta 400 usuarios sin pausa (resumen)

  THRESHOLDS
  http_req_duration
  ✓ 'p(95)<2000' p(95)=464.21ms
  http_req_failed
  ✗ 'rate<0.05' rate=57.03%
  TOTAL RESULTS
  checks_total.......: 65593  155.717811/s
  checks_succeeded...: 15.52% 10182 out of 65593
  checks_failed......: 84.47% 55411 out of 65593
  ✗ todas 200
    ↳  15% — ✓ 10182 / ✗ 55411
  HTTP
  http_req_duration..............: avg=114ms    min=167.96µs med=63.36ms  max=1.25s p(90)=334.49ms p(95)=464.21ms
    { expected_response:true }...: avg=183.81ms min=167.96µs med=102.82ms max=1.25s p(90)=485.28ms p(95)=548.55ms
  http_req_failed................: 57.03% 336694 out of 590341
  http_reqs......................: 590341 1401.469793/s
  EXECUTION
  iteration_duration.............: avg=1.32s    min=315.57ms med=1.11s    max=4.82s p(90)=2.42s    p(95)=3.11s
  iterations.....................: 65593  155.717811/s
  vus............................: 2      min=0              max=400
  vus_max........................: 400    min=400            max=400
  NETWORK
```

Figura 9. *Resumen de k6 del estrés antes de la corrección: 57 % de peticiones fallidas.*

Corrección verificada: se agregó un bloque upstream con **keepalive 64** en Nginx (reutiliza conexiones) y se repitió exactamente la misma prueba: **0 % de error con 400 usuarios** y 384 349 peticiones. Ahora el límite es la CPU de la API (~1 050 peticiones por segundo; la latencia sube, pero no falla). El cambio se probó en el contenedor local; falta subirlo al repositorio y al servidor.

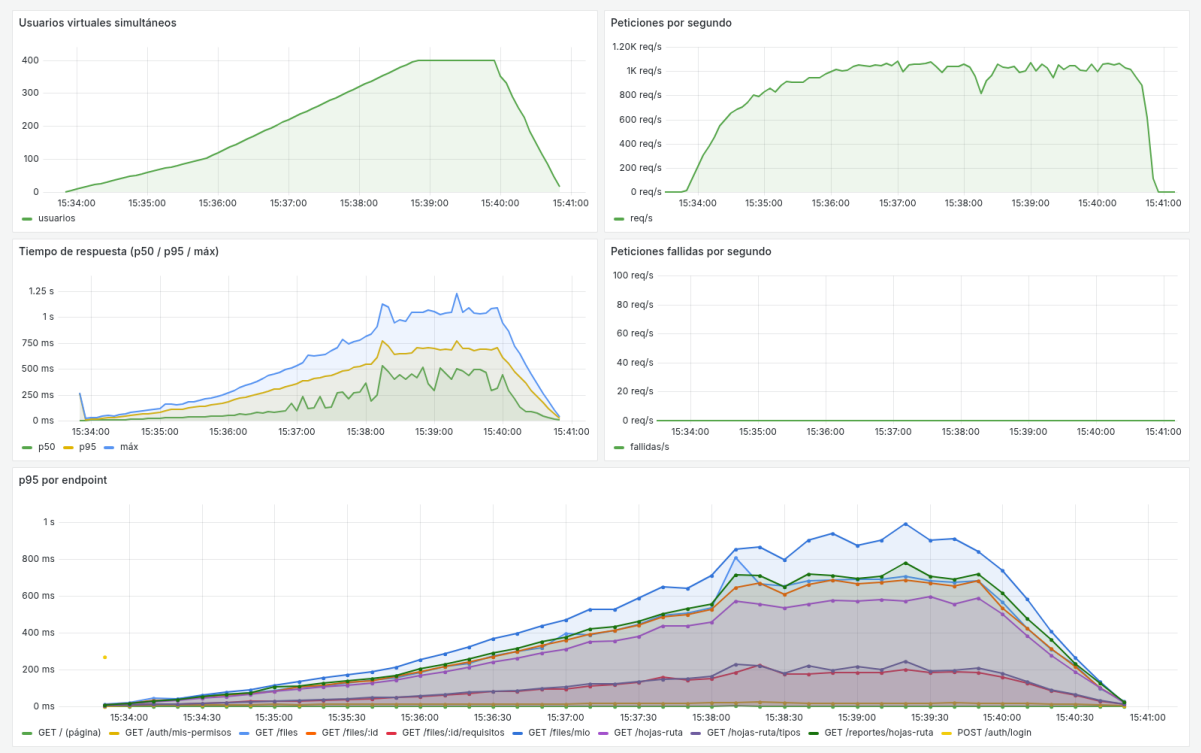


Figura 10. *Grafana — estrés DESPUÉS de la corrección: 0 peticiones fallidas hasta 400 usuarios.*



Figura 11. *Grafana — recursos durante el estrés corregido: la API usa hasta 1,7 núcleos; RAM estable.*

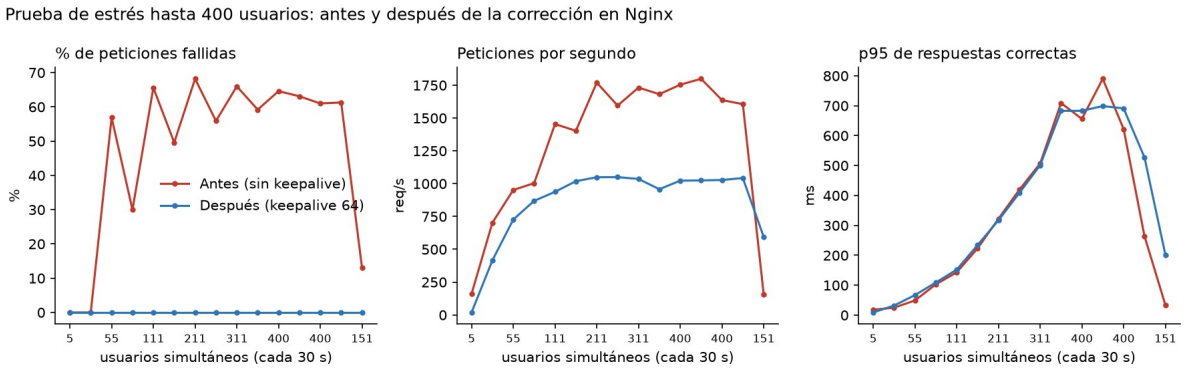


Figura 12. *Comparación del estrés antes y después de la corrección, cada 30 segundos.*

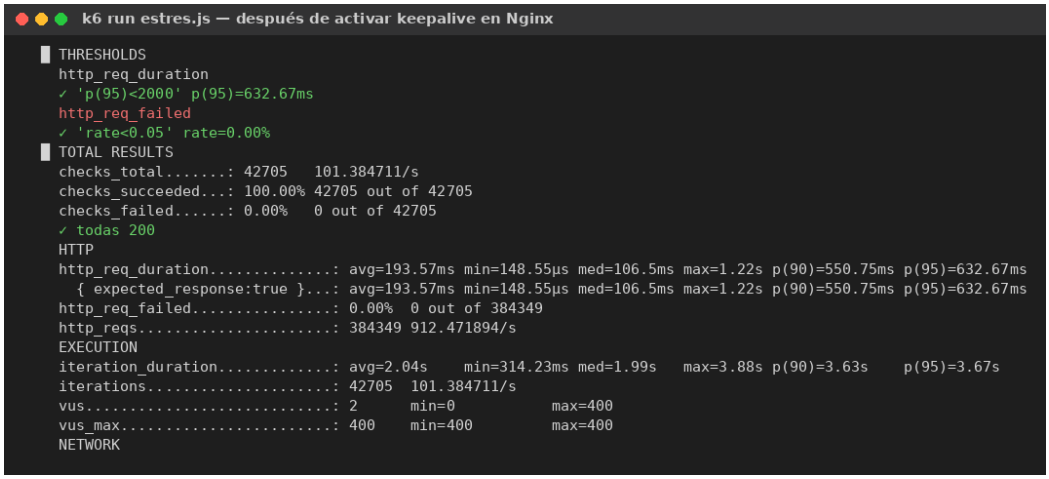


Figura 13. *Resumen de k6 del estrés después de la corrección: umbrales cumplidos.*

Interpretación de velocidad del sistema

SISET es **muy rápido** para su uso real: con 50 personas a la vez (más que todo el personal de las dos sedes), el 95 % de las respuestas tarda menos de 10 ms en el servidor, 200 veces por debajo de la meta de 2 s del proyecto. Desde el servidor de prueba en internet, una página tarda unos 450 ms (casi todo es la red). El estrés mostró un límite de configuración en Nginx, ya corregido y verificado. La capacidad sostenida es de ~1 000 peticiones por segundo.

4.2.3. Evaluación de compatibilidad

Pruebas en navegadores

Con Playwright se abrió el sistema en **Chromium**, **Google Chrome** (instalado en el equipo) y **Firefox**. En cada uno se hizo: login de Kardex, lista de files, detalle de un file y seguimiento público. Además se verificó que no hubiera barra de desplazamiento horizontal y que no hubiera errores en la consola.

Pruebas en dispositivos

Tres tamaños de pantalla en cada navegador: **escritorio 1366×768**, **tablet 820×1180** y **celular 360×800** (con pantalla táctil).

Resultados de compatibilidad

Navegador	Dispositivo	Login	Lista	Detalle	Seguimiento	Sin scroll horiz.	Carga (ms)	Resultado
Chromium	escritorio 1366×768	SÍ	SÍ	SÍ	SÍ	SÍ	655	EXITOSA
Chromium	tablet 820×1180	SÍ	SÍ	SÍ	SÍ	SÍ	647	EXITOSA
Chromium	celular 360×800	SÍ	SÍ	SÍ	SÍ	SÍ	655	EXITOSA
Google Chrome	escritorio 1366×768	SÍ	SÍ	SÍ	SÍ	SÍ	665	EXITOSA
Google Chrome	tablet 820×1180	SÍ	SÍ	SÍ	SÍ	SÍ	638	EXITOSA
Google Chrome	celular 360×800	SÍ	SÍ	SÍ	SÍ	SÍ	644	EXITOSA
Firefox	escritorio 1366×768	SÍ	SÍ	SÍ	SÍ	SÍ	829	EXITOSA

Navegador	Dispositivo	Login	Lista	Detalle	Seguimiento	Sin scroll horiz.	Carga (ms)	Resultado
Firefox	tablet 820×1180	SÍ	SÍ	SÍ	SÍ	SÍ	699	EXITOSA
Firefox	celular 360×800	SÍ	SÍ	SÍ	SÍ	SÍ	663	EXITOSA

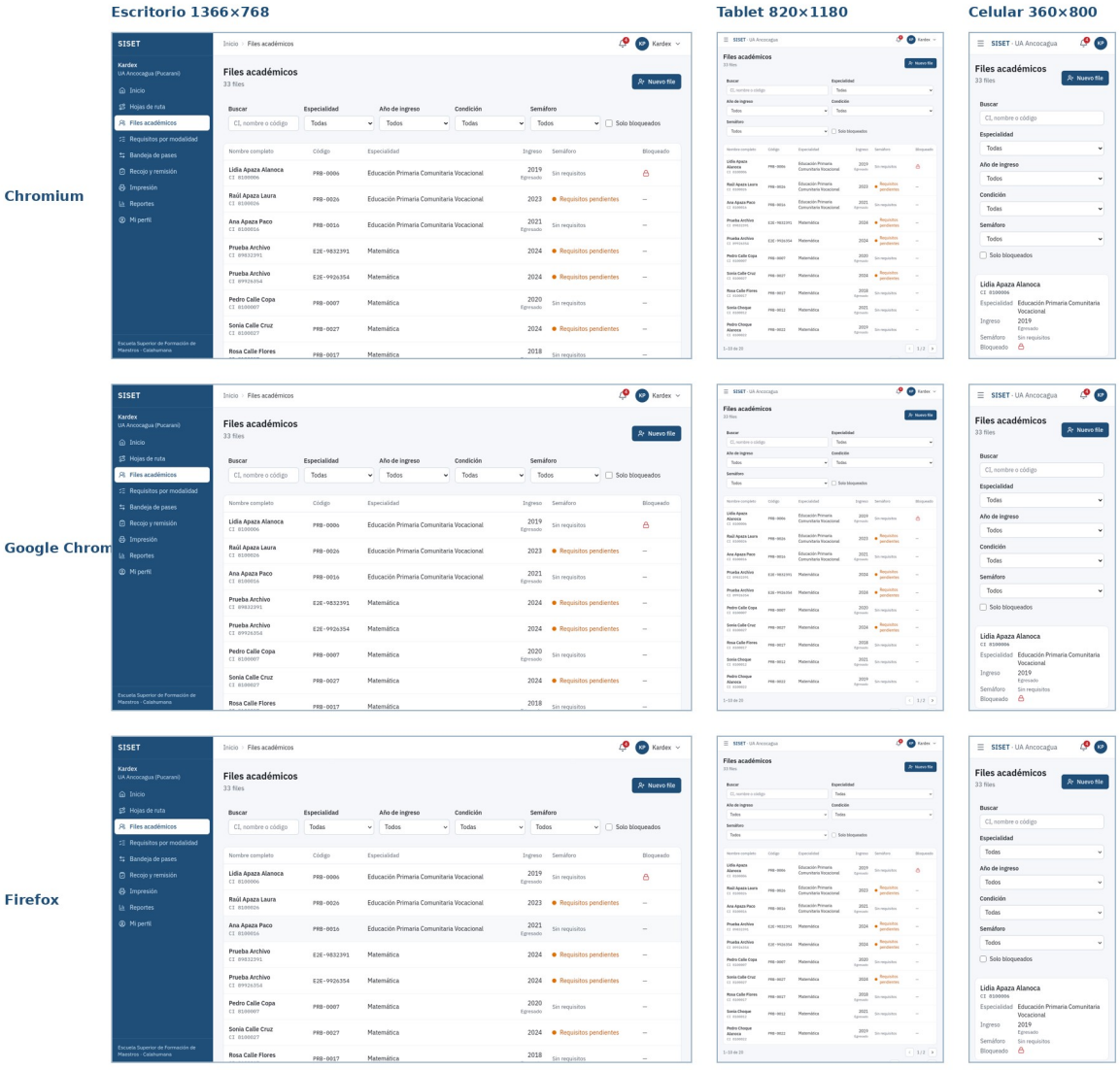


Figura 14. *Lista de files en los 3 navegadores y los 3 tamaños: en celular la tabla se convierte en tarjetas.*

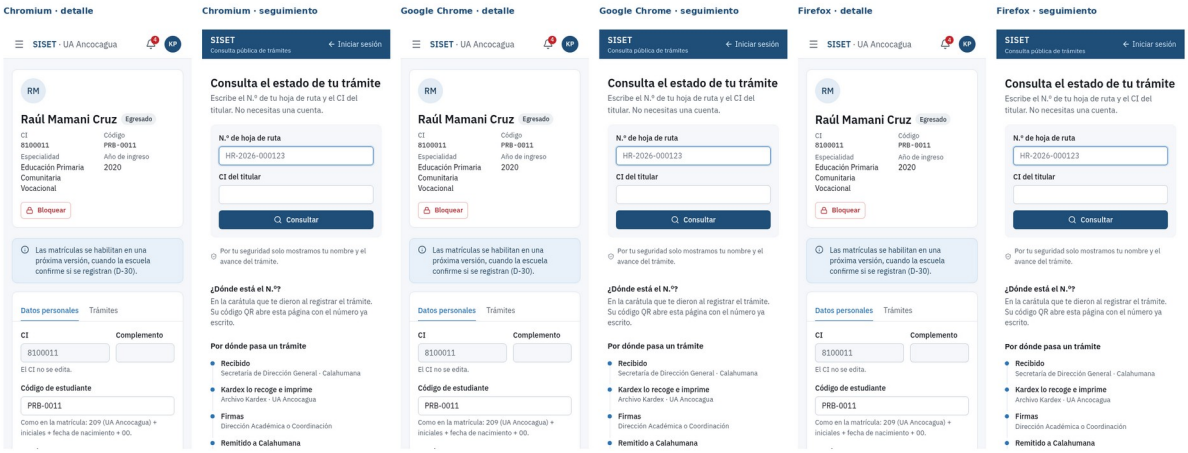


Figura 15. *Detalle del file y seguimiento público en celular (360 px) en los 3 navegadores.*

Fórmula

$$Compatibilidad = \frac{Pruebas\ Exitosas}{Pruebas\ Totales} \times 100$$

$$\text{Compatibilidad} = 9 \ / \ 9 \times 100$$

Resultado: 100 %

Interpretación

El sistema funciona igual en todos los navegadores y tamaños probados, sin errores ni desbordes. **Safari (WebKit)** no se pudo probar: este equipo Linux no tiene las librerías que necesita y Safari no existe para Linux. Queda pendiente probarlo en una Mac o un iPhone.

Nivel de compatibilidad alcanzado

100 % en los entornos evaluados (9 de 9).

Navegadores soportados

Últimas 2 versiones de **Chrome, Edge** (mismo motor que Chrome), **Firefox** y **Safari** (requisito del proyecto). Probados hoy: Chromium, Google Chrome y Firefox.

4.2.4. Evaluación de usabilidad

Encuestas aplicadas

> **Aún no se aplicaron encuestas a usuarios reales.** Se aplicarán en la capacitación del 28–29 de octubre con el cuestionario **SUS** (System Usability Scale, 10 preguntas, 0 a 100 puntos). Por eso aquí se usa una medición objetiva de accesibilidad con **Lighthouse** (Google), que sí se pudo hacer hoy. No se inventan resultados de encuestas.

Evaluación de experiencia de usuario

- Responsivo desde 360 px sin scroll horizontal (comprobado en 4.2.3).
- Menú según el rol: cada persona ve solo lo que usa.
- Inicio de cada rol con indicadores y el camino del trámite.
- Mensajes de error en español, con el campo señalado.
- Manual y guía de capacitación dentro del sistema.

Resultados estadísticos

Página	Accesibilidad	Buenas prácticas	Rendimiento	SEO
/login	98	96	86	82
/seguimiento (público)	100	96	87	82
/guia/manual	100	96	77	82
Promedio	99,33	96,00	83,33	82,00

Fórmula

$$Usabilidad = \frac{Puntaje\ Obtenido}{Puntaje\ Máximo} \times 100$$

$$\text{Usabilidad (accesibilidad)} = 298 \ / \ 300 \times 100$$

Resultado: 99,33 %

Contenido — Tablas de encuestas

N.º	Pregunta SUS (a aplicar el 28–29 oct)	Escala
1	Creo que me gustaría usar este sistema con frecuencia.	1 a 5
2	Encontré el sistema innecesariamente complejo.	1 a 5
3	Pensé que el sistema era fácil de usar.	1 a 5
4	Creo que necesitaría ayuda de un técnico para usarlo.	1 a 5
5	Las funciones del sistema están bien integradas.	1 a 5
6	Hay demasiada inconsistencia en el sistema.	1 a 5
7	La mayoría de las personas aprendería a usarlo muy rápido.	1 a 5
8	El sistema me pareció muy incómodo de usar.	1 a 5
9	Me sentí muy seguro usando el sistema.	1 a 5
10	Necesité aprender muchas cosas antes de poder usarlo.	1 a 5

Cálculo SUS: preguntas impares (respuesta − 1), pares (5 − respuesta), suma × 2,5 = puntaje de 0 a 100. Usabilidad = puntaje obtenido / 100 × 100.

Gráficos estadísticos

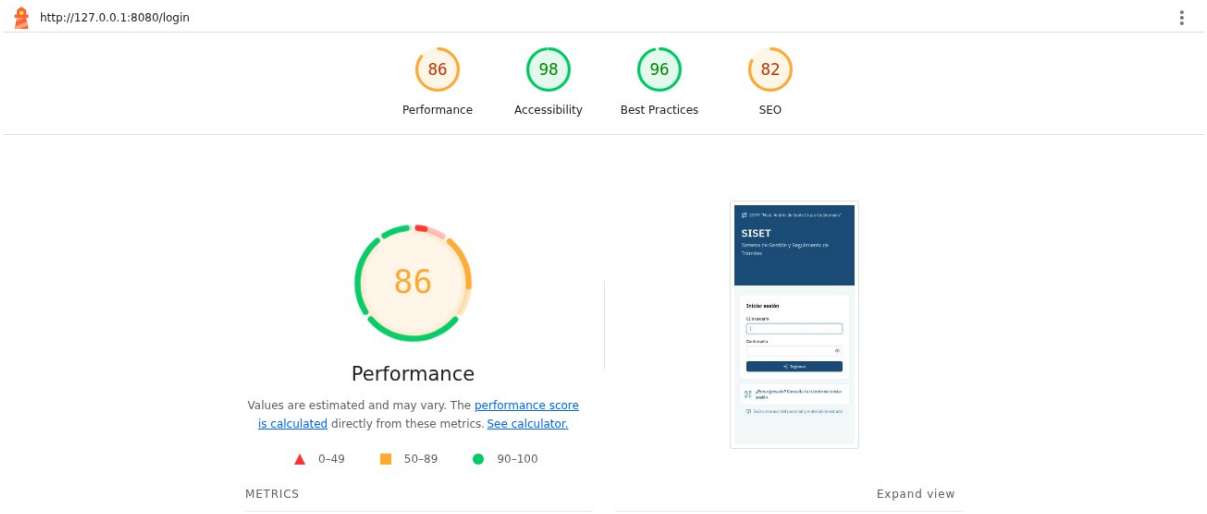


Figura 16. *Lighthouse en /login: accesibilidad 98, buenas prácticas 96.*

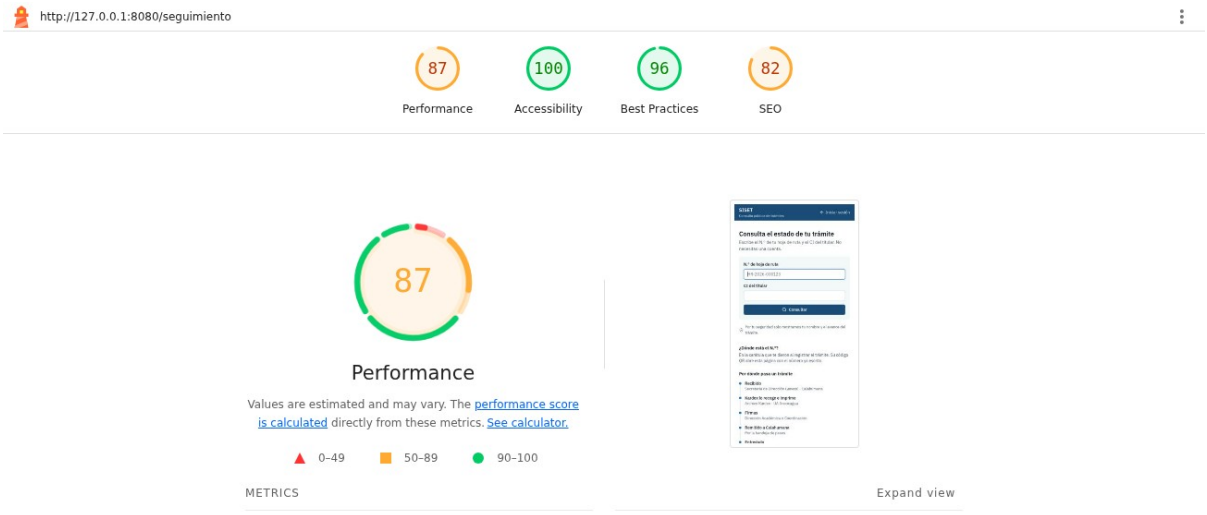


Figura 17. *Lighthouse en el seguimiento público: accesibilidad 100.*

Interpretación del nivel de satisfacción

La accesibilidad técnica es **excelente (99,33 %)**: contraste, etiquetas en los formularios, navegación con teclado y tamaños táctiles. La satisfacción de los usuarios se medirá con SUS en la capacitación; un puntaje SUS mayor a 68 se considera aceptable y mayor a 80, bueno.

4.2.5. Evaluación de fiabilidad

Registro de errores

El servidor de prueba registra cada petición en logs estructurados (pino). Desde el último despliegue: **15 522 respuestas**, de las cuales **0 son errores del servidor (5xx)** y 0 son logs de nivel error. Las 9 respuestas 4xx son esperadas (401 sin sesión, 409 por un dato duplicado, 429 por exceso de logins, 400 por datos inválidos).

Fallos detectados

Fallo	Dónde	Causa	Estado
502 con más de ~60 usuarios sin pausa	Estrés local	Nginx sin keepalive hacia la API	Corrección verificada; falta subirla

Fallo	Dónde	Causa	Estado
1 prueba E2E intermitente	Playwright (mock)	Espera de un campo en la prueba, no en el sistema	Pendiente
La web no envía HSTS (la API sí)	Servidor de prueba	Falta la cabecera en Nginx	Pendiente
16 ejecuciones de CI fallidas en main	GitHub Actions	Cambios que rompían pruebas o formato	Corregidas (4 de 4 rachas)

Disponibilidad del sistema

Sondeo del servidor público cada 15 segundos durante 10 minutos (página y API): **80 de 80 respuestas correctas**. Los contenedores llevan **44 horas en línea sin reinicios** y el servidor, 3 días. Durante el estrés local la API tampoco se cayó.

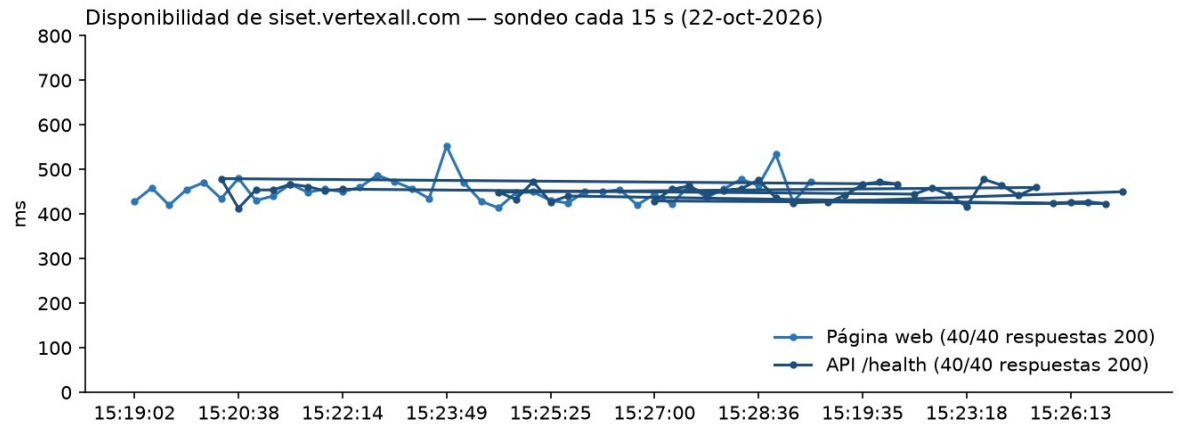


Figura 18. *Disponibilidad de siset.vertexall.com: 80/80 respuestas 200, unos 450 ms por respuesta.*

Fórmula

$$Disponibilidad = \frac{Tiempo\ Operativo}{Tiempo\ Total} \times 100$$

$$\text{Disponibilidad} = 44\ \text{h} \ / \ 44\ \text{h} \times 100 \quad \cdot \quad \text{sondeo: } 80 \ / \ 80 \times 100$$

Resultado: 100 %

Integridad de datos bajo concurrencia. 30 usuarios crearon **300 hojas de ruta al mismo tiempo**. Luego se revisó la base: ningún número de trámite repetido, ningún salto en el correlativo, cada hoja con su movimiento inicial y su evento en la bitácora, y ningún registro huérfano.

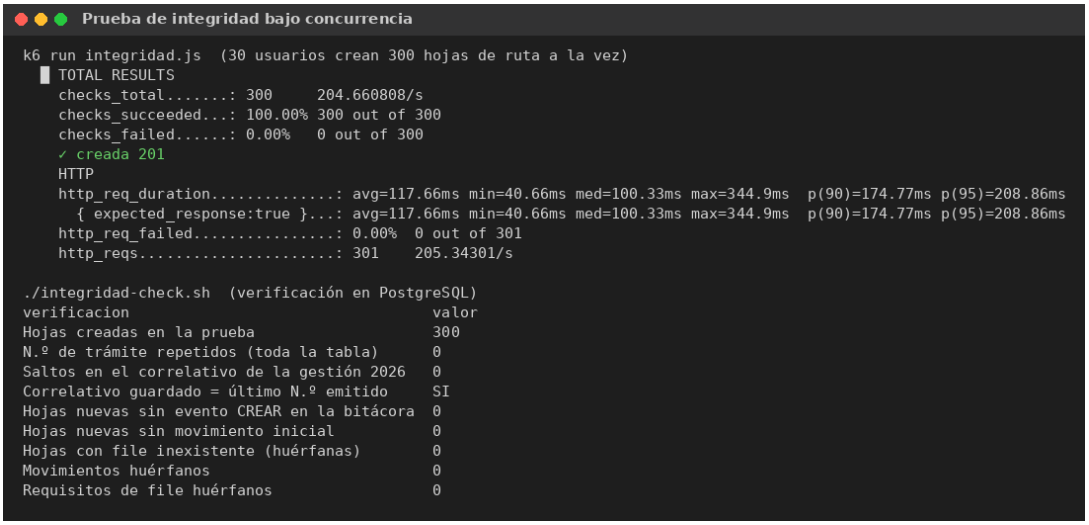


Figura 19. *Prueba de integridad: 300 altas simultáneas y 9 verificaciones en PostgreSQL, todas correctas.*

Interpretación

El sistema es **estable**: no tuvo caídas ni errores del servidor en uso real, y mantiene los datos consistentes aunque muchas personas escriban a la vez (el correlativo se genera dentro de una transacción con bloqueo de fila).

Estabilidad del sistema

Alta: 0 reinicios, 0 errores 5xx, memoria estable bajo carga (la API se mantiene en ~180–230 MB).

Nivel de disponibilidad alcanzado

100 % en el periodo medido (44 h). La meta para producción es ≥ 99 % mensual.

4.2.6. Evaluación de seguridad

Validación de accesos

Se probó contra la API: peticiones sin token, con token alterado o falso, roles que intentan entrar donde no deben (SEC_DIR a files, el estudiante al file de otro, Kardex a usuarios). Todas fueron rechazadas (401 o 403).

Gestión de sesiones

Token de acceso de 15 minutos, renovación en cookie httpOnly y Secure, cierre de sesión que invalida la renovación y **detección de reuso** (en el servidor de prueba se detectaron y cortaron 3 intentos de reuso).

Protección de datos

Contraseñas con bcrypt 12, validación estricta de la entrada (inyección SQL sin efecto, campos extra rechazados, cuerpos grandes rechazados), bitácora inmutable y HTTPS.

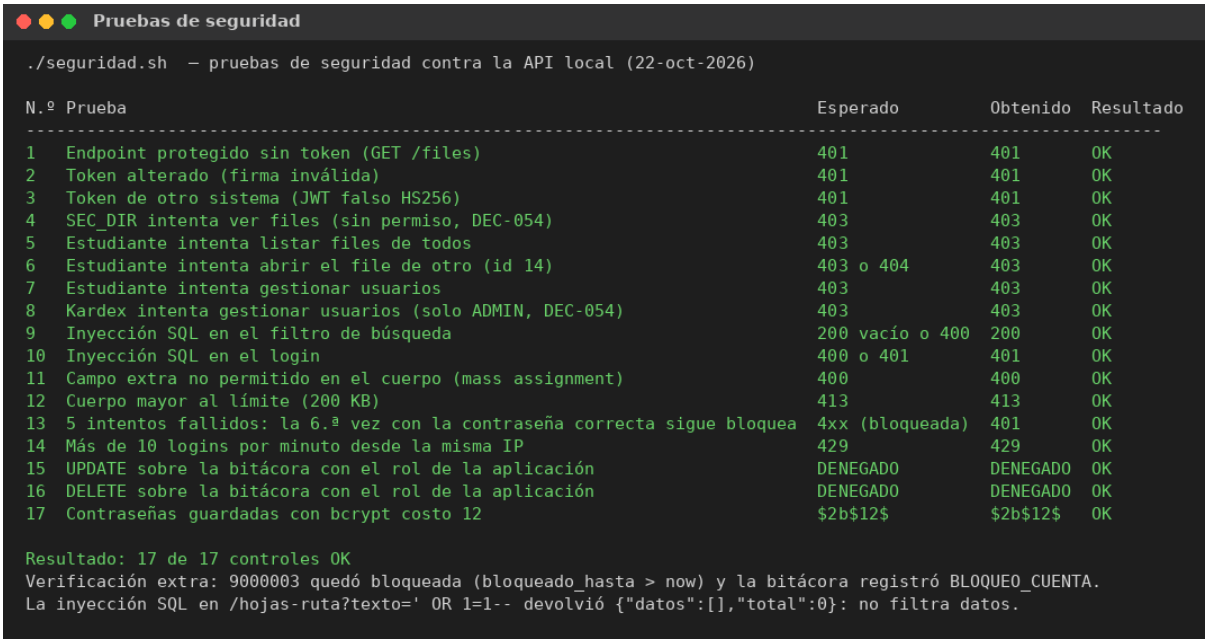


Figura 20. *Pruebas de seguridad: 17 de 17 controles OK.*

Fórmula

$$Seguridad = \frac{Controles\ Implementados}{Controles\ Requeridos} \times 100$$

$$\text{Seguridad} = 20 \ / \ 24 \times 100$$

Resultado: 83,33 %

Resultados — Tabla de controles aplicados

N.º	Control requerido	¿Implementado?
1	Contraseñas con bcrypt costo ≥ 12	Sí
2	JWT de 15 min	Sí
3	Renovación en cookie httpOnly, Secure, SameSite	Sí
4	Logout invalida la sesión	Sí
5	Bloqueo tras 5 intentos	Sí
6	Límite de logins por IP	Sí
7	Permisos por rol (RBAC)	Sí
8	Cada usuario ve solo lo suyo y lo de su sede	Sí
9	Validación de entrada (Zod)	Sí
10	Cabeceras de seguridad (Helmet, CSP)	Sí
11	CORS restringido	Sí
12	HTTPS con redirección	Sí
13	HSTS en todas las respuestas	No (solo en la API)
14	Bitácora de escrituras y accesos	Sí
15	Bitácora inmutable	Sí
16	Rol de base con mínimo privilegio	Sí
17	Detección de reuso de sesión	Sí
18	Doble verificación (TOTP)	No (pendiente)

N.º	Control requerido	¿Implementado?
19	Respaldo diario automático	No (pendiente)
20	Límite de tamaño de las peticiones	Sí
21	Auditoría de dependencias en CI	Sí
22	Contenedores sin privilegios	Sí
23	Cambio obligatorio de contraseña temporal	Sí
24	Cifrado de la base en reposo	No

Nivel de cumplimiento de seguridad

83,33 % (20 de 24). Los 4 faltantes son conocidos y tienen tarea: HSTS en la web (una línea en Nginx), TOTP, respaldo diario y cifrado en reposo.

4.2.7. Evaluación de mantenibilidad

Tiempo de corrección de errores

En la rama principal hubo 4 rachas de integración fallida; todas se corrigieron, con una **mediana de 17 minutos** (promedio 46 min, máximo 2,5 h). El fallo de estrés de hoy se diagnosticó y se verificó su corrección en unos 15 minutos.

Modularidad del sistema

Monolito modular con **7 módulos** (seguridad, personas, catálogos, files, hojas-ruta, auditoría, salud), cada uno con capas rutas → controlador → servicio → repositorio. Un módulo solo usa a otro por su index.ts y cada tabla tiene un módulo dueño.

Facilidad de actualización

Cambios por migraciones versionadas (17), imágenes Docker etiquetadas por fecha y commit, despliegue con un clic en Coolify y vuelta atrás cambiando la etiqueta. 1 490 pruebas automáticas avisan si un cambio rompe algo.

Fórmula

$$Mantenibilidad = \frac{Cambios\ Realizados\ Exitosamente}{Cambios\ Solicitados} \times 100$$

$$\text{Mantenibilidad} = 63 \ / \ 79 \times 100 \quad \text{(integraciones a main que pasaron la CI al primer intento)}$$

Resultado: 79,75 %

Además, el **100 %** de los cambios fallidos se corrigió (4 de 4). Cobertura de pruebas del módulo de seguridad: 98,55 % (meta ≥ 70 %).

4.2.8. Evaluación de portabilidad

Instalación en diferentes entornos

Entorno	Sistema	Resultado
Desarrollo local	Linux + Node 26 + PostgreSQL en Docker	Funciona
Stack completo local	Docker Compose (db, migrar, api, web)	Funciona (usado en estas pruebas)
Integración continua	GitHub Actions (runner Ubuntu)	Funciona (migraciones, E2E, Docker)

Entorno	Sistema	Resultado
Servidor de prueba	VPS Contabo + Coolify + Traefik	Funciona (44 h en línea)

Compatibilidad de despliegue

Las mismas imágenes Docker corren en cualquier servidor con Docker. No dependen de servicios de un proveedor en particular. Se podrían instalar en un servidor de la escuela.

Fórmula

$$Portabilidad = \frac{Entornos\ Compatibles}{Entornos\ Evaluados} \times 100$$

$$\text{Portabilidad} = 4 \ / \ 4 \times 100$$

Resultado: 100 %

4.3. Evaluación de seguridad — ISO/IEC 27001

4.3.1. Evaluación de riesgos

Identificación de amenazas

Se identificaron 13 amenazas a partir de la arquitectura, los resultados de las pruebas y las dudas abiertas con la escuela.

Probabilidad e impacto

Escala de 1 a 5. Probabilidad: 1 rara … 5 casi segura. Impacto: 1 mínimo … 5 crítico (pérdida de datos o acceso indebido).

Fórmula de riesgo

$$Riesgo = Probabilidad \times Impacto$$

$$\text{Ejemplo R1: Riesgo} = 3 \times 5 = 15$$

Resultado: Bajo 1–6 · Medio 8–12 · Alto 15–25

Resultados — Tabla de riesgos

ID	Amenaza	P	I	Riesgo	Nivel	Control / tratamiento
R1	Pérdida de datos: aún no hay respaldo automático diario en producción	3	5	15	ALTO	Respaldo diario con pg_dump y retención de 30 días antes de la puesta en producción; restauración ya probada en local (sección 4.3.2)
R2	Caída del VPS o de Coolify	2	4	8	MEDIO	Contenedores con reinicio automático y healthchecks; volver a una versión anterior por etiqueta de imagen
R3	Corte de internet en la sede Ancocagua	4	3	12	MEDIO	La interfaz avisa “sin conexión” y no pierde formularios; el trámite sigue en papel y se registra después
R4	Robo o adivinación de contraseñas	2	5	10	MEDIO	bcrypt costo 12, bloqueo tras 5 intentos, límite de 10 logins/min por IP; falta la doble verificación TOTP
R5	Un usuario ve datos de otro rol o de otro estudiante	1	5	5	BAJO	Casbin por rol, “propio” y sede validados en el servicio; probado (sección 4.3.3)
R6	Saturación de la web con mucho tráfico (hallazgo del estrés)	2	3	6	BAJO	Activar keepalive entre Nginx y la API (verificado: ver 4.2.2)
R7	Reglas del negocio que la escuela aún no confirma	3	3	9	MEDIO	Registro de dudas y decisiones (DEC); reglas provisionales señaladas
R8	Dependencia con vulnerabilidad conocida	2	4	8	MEDIO	npm audit en cada CI (0 altas y críticas); overrides cuando hace falta (csv-parse)
R9	Alteración o borrado de la bitácora	1	4	4	BAJO	Rol de BD de la app sin UPDATE/DELETE/TRUNCATE + triggers de solo inserción
R10	Error humano al cargar datos	3	2	6	BAJO	Validación con Zod, confirmaciones y bitácora con valores antes/después
R11	Poca adopción por falta de capacitación	3	3	9	MEDIO	Capacitación 28–29 oct, manual en /guia/manual
R12	Un solo responsable técnico del sistema	3	4	12	MEDIO	Documentación completa en docs/ (decisiones, diseño por tarea, guía de despliegue)

ID	Amenaza	P	I	Riesgo	Nivel	Control / tratamiento
R13	Vencimiento del certificado HTTPS	1	3	3	BAJO	Renovación automática de Let's Encrypt con Traefik

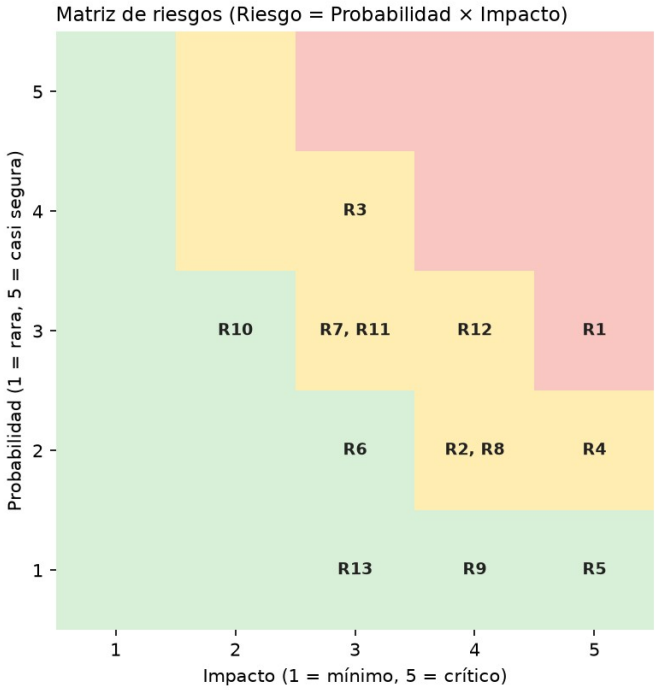


Figura 21. *Matriz de riesgos: verde bajo, amarillo medio, rojo alto.*

Riesgos altos, medios y bajos

Alto (1): R1, la pérdida de datos por falta de respaldo diario. **Medios (7):** R3, R12, R4, R7, R11, R2 y R8. **Bajos (5):** R6, R10, R5, R9 y R13.

Interpretación

El único riesgo alto (R1) se resuelve antes de la puesta en producción con el respaldo diario, cuyo procedimiento ya se probó (4.3.2). Los riesgos de acceso indebido son bajos gracias a los controles probados. El riesgo R6 (saturación) bajó a bajo con la corrección verificada.

4.3.2. Evaluación de copias de seguridad

Frecuencia de backups

Planificada: diaria, con retención de 30 días. **Hoy no hay respaldo automático en el servidor** (pendiente antes de la puesta en producción). Se probó el procedimiento manual.

Recuperación de información

Se generó un respaldo con **pg_dump** (formato custom, 110 KB, 0,2 s) y se restauró con **pg_restore** en una base nueva (0,5 s). Luego se compararon las filas de cada tabla y una huella MD5 de las hojas de ruta.

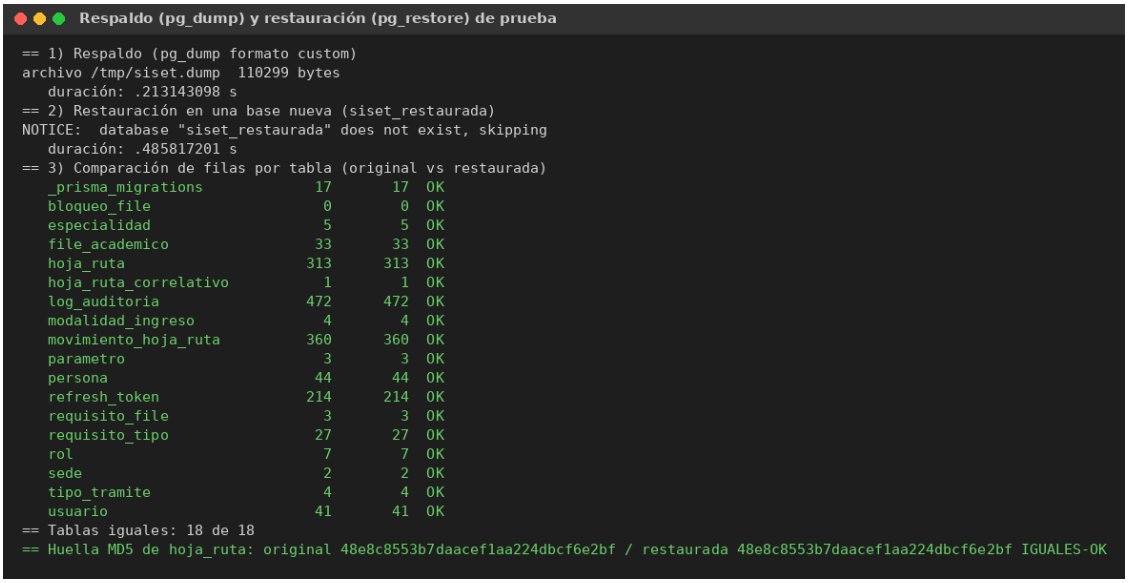


Figura 22. *Respaldo y restauración: 18 de 18 tablas con las mismas filas y la misma huella MD5.*

Resultados de restauración

18 de 18 tablas idénticas (incluidas 313 hojas de ruta, 360 movimientos y 472 eventos de bitácora) y huella MD5 igual.

Fórmula

$$Tasa\ de\ Recuperación = \frac{Backups\ Recuperados}{Backups\ Realizados} \times 100$$

$$\text{Tasa de recuperación} = 1 \ / \ 1 \times 100 \quad (18 \ / \ 18 \text{ tablas})$$

Resultado: 100 %

4.3.3. Evaluación de seguridad de acceso

Usuarios autenticados

Bitácora del servidor de prueba (15 al 21 de octubre de 2026): **113 inicios de sesión válidos**, 61 cierres de sesión, **10 intentos fallidos** (rechazados) y 3 reusos de sesión detectados y cortados.

Intentos fallidos

Los 10 intentos fallidos se rechazaron y quedaron registrados con IP y hora. En la prueba local, 5 intentos seguidos **bloquearon la cuenta** y quedó el evento BLOQUEO_CUENTA; con más de 10 logins por minuto desde una IP, la API respondió **429**.

Control de permisos

Cada rol tiene su política en Casbin; se probaron 7 intentos de acceso indebido y todos fueron rechazados (ver la Figura de seguridad).

Fórmula

$$Accesos\ Seguros = \frac{Accesos\ Válidos}{Accesos\ Totales} \times 100$$

$$\text{Accesos seguros} = 113 \ / \ 123 \times 100$$

Resultado: 91,87 % de los intentos fueron accesos válidos; el 100 % de los inválidos fue rechazado

4.3.4. Evaluación de vulnerabilidades

Pruebas de seguridad realizadas

- npm audit (todas las dependencias y solo las de producción).
- 17 pruebas de ataque contra la API (tokens, permisos, inyección SQL, mass assignment, fuerza bruta, bitácora).
- Revisión de cabeceras HTTP y del certificado TLS.
- Prueba de estrés (resistencia a saturación).

Vulnerabilidades encontradas

N.º	Vulnerabilidad	Severidad	Estado
1	csv-parse 5.x (GHSA-8cw4-87c7-c6xx), dependencia de Casbin	Moderada	Corregida (override a csv-parse 7)
2	Saturación de conexiones Nginx → API con tráfico alto	Media	Corrección verificada en local; falta subirla
3	La web no envía HSTS	Baja	Pendiente (una línea en Nginx)

Vulnerabilidades corregidas

1 de 3 corregida en el servidor. Las otras 2 tienen la corrección identificada y son de bajo esfuerzo.

Fórmula

$$Corrección\ de\ Vulnerabilidades = \frac{Vulnerabilidades\ Corregidas}{Vulnerabilidades\ Detectadas} \times 100$$

$$\text{Corrección de vulnerabilidades} = 1 \ / \ 3 \times 100$$

Resultado: 33,33 % (sube a 100 % al subir las 2 correcciones)

4.4. Resultados generales

Tabla consolidada de métricas

Norma	Característica / métrica	Fórmula aplicada	Resultado
ISO 25010	Adecuación funcional	103 / 104 × 100	99,04 %
ISO 25010	Eficiencia de desempeño	p95 = 9,35 ms < 2 s (meta); CPU 6,4 %	100 % de la meta
ISO 25010	Compatibilidad	9 / 9 × 100	100 %
ISO 25010	Usabilidad (accesibilidad)	298 / 300 × 100	99,33 %
ISO 25010	Fiabilidad (disponibilidad)	44 h / 44 h × 100	100 %
ISO 25010	Seguridad	20 / 24 × 100	83,33 %
ISO 25010	Mantenibilidad	63 / 79 × 100	79,75 %
ISO 25010	Portabilidad	4 / 4 × 100	100 %
ISO 27001	Riesgo	P × I	1 alto · 7 medios · 5 bajos
ISO 27001	Tasa de recuperación	1 / 1 × 100	100 %
ISO 27001	Accesos seguros	113 / 123 × 100	91,87 %
ISO 27001	Corrección de vulnerabilidades	1 / 3 × 100	33,33 %

Gráficos comparativos

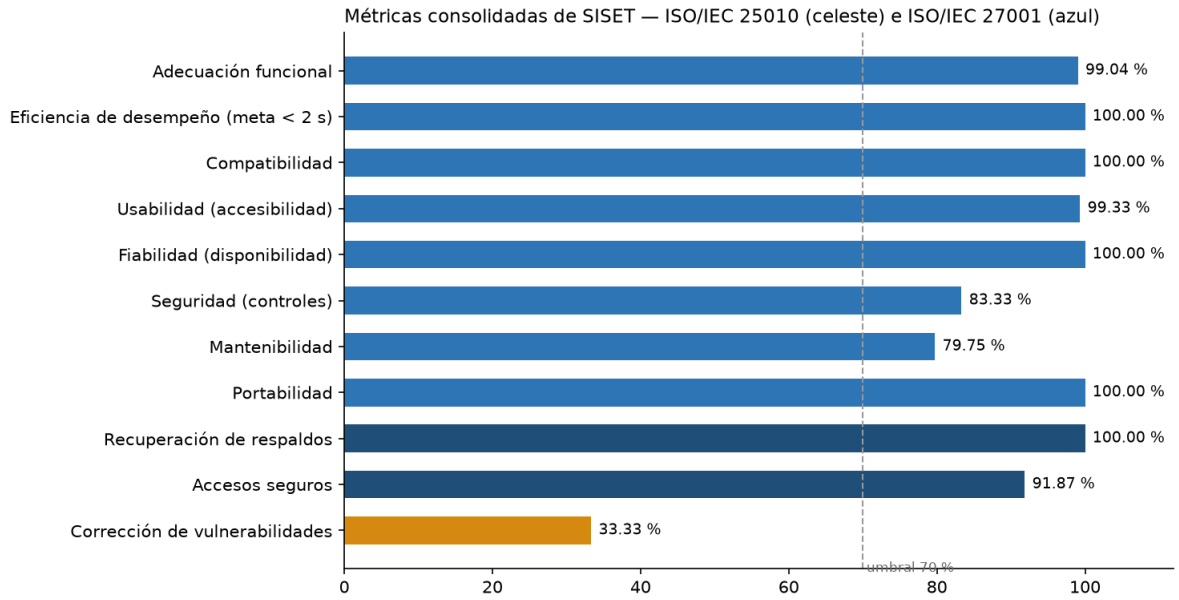


Figura 23. *Métricas consolidadas: todas superan el 70 %, salvo la corrección de vulnerabilidades (2 pendientes).*

Cumplimiento de ISO 25010

Promedio de las 8 características: (99,04 + 100 + 100 + 99,33 + 100 + 83,33 + 79,75 + 100) / 8 = **95,18 %**. Nivel: **alto**.

Cumplimiento de ISO 27001

Promedio de las métricas porcentuales (recuperación 100, accesos 91,87, vulnerabilidades 33,33 y controles 83,33) = **77,13 %**. Controles del Anexo A revisados: 8 cumplen y 3 en forma parcial. Nivel: **aceptable**, con un plan claro para llegar a alto.

Interpretación global de resultados

SISET cumple con holgura los atributos de calidad: funciona, es rápido, compatible, accesible, estable y portable. En seguridad, los controles de acceso y la protección de datos están probados. Lo que baja el puntaje son tareas conocidas y planificadas (respaldo diario, TOTP, HSTS y keepalive), no fallas de diseño.

4.5. Conclusiones del capítulo

Nivel de calidad alcanzado

95,18 % según ISO/IEC 25010 (alto). Con 50 usuarios simultáneos responde en milisegundos, sin errores, y funciona en 3 navegadores y 3 tamaños de pantalla.

Nivel de seguridad alcanzado

77,13 % según ISO/IEC 27001 (aceptable). 17/17 pruebas de ataque rechazadas, bitácora inmutable y 0 vulnerabilidades en las dependencias. Faltan 4 controles conocidos.

Fortalezas del sistema

- Rendimiento: p95 de 9,35 ms con 50 usuarios.
- Integridad: correlativo sin duplicados ni saltos con 300 altas simultáneas.
- Seguridad de acceso por rol probada y trazabilidad completa.
- 1 490 pruebas automáticas y CI con puertas de calidad.
- Despliegue reproducible con Docker y vuelta atrás inmediata.
- Accesible (98–100) y responsivo desde 360 px.

Aspectos a mejorar

- Programar el respaldo diario automático con retención de 30 días (riesgo alto R1).
- Subir la corrección de keepalive en Nginx y agregar HSTS en la web.
- Implementar la doble verificación TOTP.
- Cifrar el volumen de la base y los respaldos.
- Plantillas oficiales y exportar reportes a PDF/Excel (vista previa, impresión oficial y reportes).
- Aplicar la encuesta SUS en la capacitación y probar en Safari.
- Corregir la prueba E2E intermitente.

Validación del cumplimiento de estándares ISO

Las 13 fórmulas del archivo de observaciones se calcularon con datos medidos y con evidencia (capturas de Grafana, k6, Playwright, Lighthouse, PostgreSQL y del servidor). Con ISO/IEC 25010 el producto alcanza un nivel alto. Con ISO/IEC 27001 alcanza un nivel aceptable y tiene un plan concreto, de bajo esfuerzo, para cerrar las brechas antes de la puesta en producción.

Glosario de términos técnicos

Término	Qué significa
p95 (percentil 95)	El 95 % de las respuestas tardó menos que ese valor. Es más exigente que el promedio porque muestra los casos lentos.
ms	Milisegundo: la milésima parte de un segundo.
Prueba de carga	Simula muchos usuarios a la vez durante un uso normal para medir la velocidad.
Prueba de estrés	Sube los usuarios hasta que el sistema falle, para conocer su límite.
E2E (de punta a punta)	Prueba automática que usa el sistema como una persona: abre el navegador, hace clic y escribe.
CI (integración continua)	Cada cambio en el código se prueba automáticamente antes de aceptarlo.
k6	Herramienta libre para pruebas de carga y estrés.
Grafana / InfluxDB	Grafana dibuja los gráficos en tiempo real; InfluxDB guarda las mediciones.
Playwright	Herramienta que controla navegadores (Chrome, Firefox) para pruebas automáticas.
Lighthouse	Herramienta de Google que califica de 0 a 100 la accesibilidad, el rendimiento y las buenas prácticas de una página.
SUS	Cuestionario estándar de 10 preguntas que mide la facilidad de uso de un sistema (0 a 100 puntos).
Doble verificación (TOTP)	Además de la contraseña, se pide un código de 6 dígitos que genera una aplicación en el celular.
HSTS	Cabecera que obliga al navegador a usar siempre la conexión segura (https).
Keepalive	Reutilizar una misma conexión de red para muchas peticiones en vez de abrir una nueva cada vez.
Nginx	Servidor web que entrega las páginas y pasa las consultas a la API.
API	La parte del sistema que recibe las consultas, aplica las reglas y lee o guarda en la base de datos.
bcrypt	Forma segura de guardar contraseñas: se guarda una “huella” que no se puede revertir.
JWT	Credencial digital firmada que identifica al usuario durante 15 minutos.
Errores 401 / 403 / 429 / 5xx	401: no inició sesión; 403: no tiene permiso; 429: demasiados intentos; 5xx: error del servidor.
Docker / contenedor	Forma de empaquetar el sistema para que funcione igual en cualquier servidor.
VPS / Coolify	VPS: servidor virtual alquilado en internet. Coolify: panel para instalar y actualizar el sistema en él.
pg_dump / pg_restore	Herramientas de PostgreSQL para sacar una copia de seguridad de la base y para restaurarla.
Correlativo	Número de trámite consecutivo y único por año (por ejemplo, HR-2026-000123).
Bitácora de auditoría	Registro de quién hizo qué, cuándo y desde dónde; no se puede modificar ni borrar.
KLOC	Miles de líneas de código fuente (se usa en el modelo COCOMO).

Anexo — Cómo repetir las pruebas

Prueba	Comando
Levantar SISET	VITE_USAR_API=true docker compose --profile completo up -d --build --wait
Grafana + InfluxDB	docker compose up -d (carpeta grafana; Grafana en http://localhost:3300)
Carga	k6 run --out influxdb=http://127.0.0.1:8086/k6 carga.js
Estrés	k6 run --out influxdb=http://127.0.0.1:8086/k6 estres.js
Integridad	k6 run integridad.js && ./integridad-check.sh <id_base>
Seguridad	./seguridad.sh
Compatibilidad	node compatibilidad.mjs
Respaldo	./respaldo.sh
Unitarias e integración	npm test · npm run test:cobertura -w @siset/api
E2E	E2E_API_REAL=1 npx playwright test (apps/web)